Highlights

A Hybrid Edge-Cloud Stream Processing Framework for Low-Latency Real-Time Analytics

Umur Inan

- HybridStream achieves 99.2–99.8% SLO compliance for latency-critical stream operators, a 2.4–6.8 percentage-point improvement over static hybrid baselines ($p < 0.001$).
- The AODE continuously recalibrates operator placement within 18–36 ms, consuming less than 2% CPU overhead.
- The PCTR protocol migrates stateful operators with bounded pause times of 62–1,410 ms, with zero SLO violations attributable to migration across 630 experiment runs.
- The lightweight edge runtime reduces memory footprint by 95.3% compared to Apache Flink TaskManager (55 MB vs 1,187 MB).
- Experimental evaluation across three workloads and three network profiles demonstrates adaptive placement sustains peak throughput within 5.7% of static baselines.

A Hybrid Edge-Cloud Stream Processing Framework for Low-Latency Real-Time Analytics

Umur Inan[a,*]

[a] *Solarity AI, United States*

Abstract

Real-time stream processing applications increasingly impose sub-10-ms service-level objectives (SLOs) on individual operators while simultaneously requiring the compute scalability of cloud infrastructure for bulk analytics. Neither cloud-only nor static hybrid deployments satisfy both constraints: cloud-only architectures incur irreducible WAN latency that structurally violates millisecond SLOs, while static hybrid systems lack mechanisms to respond to load spikes, memory pressure, or ingest rate variability at runtime. This paper presents HybridStream, a hybrid edge-cloud stream processing framework in which an Adaptive Offloading Decision Engine (AODE) continuously recalibrates operator placement across edge and cloud tiers. The AODE employs a multi-factor scoring model that integrates EWMA-smoothed latency estimates, resource utilization, SLO urgency, and inter-tier transfer costs into a single placement quality measure. Stateful operators are migrated between tiers at runtime via the pause-checkpoint-transfer-resume (PCTR) protocol, which bounds migration pause time to 62–1,410 ms depending on operator state size. The HybridStream Edge Agent (HEA) provides a lightweight

*Corresponding author
Email address: umur@solarity.ai (Umur Inan)

Python 3.12 execution environment with a 55.2 MB baseline footprint,a 95.3% reduction relative to an Apache Flink TaskManager,enabling dense co-hosting on commodity edge hardware. Experimental evaluation across three representative workloads (industrial IoT, smart city, financial processing) and three network profiles (10–150 ms RTT) demonstrates that HybridStream achieves 99.2–99.8% SLO compliance,a 2.4–6.8 percentage-point improvement over a statically partitioned hybrid baseline ($p < 0.001$, rank-biserial $r = 0.88$–0.94),while sustaining peak throughput within 5.7% of the static baseline. AODE recalibration overhead remains below 2% CPU and 145 MB RSS on a 2-vCPU host, with placement decisions issued within 36 ms.

Keywords: stream processing, edge computing, cloud computing, operator placement, adaptive offloading, stateful migration, SLO compliance, hybrid architecture

1. Introduction

The proliferation of Internet of Things (IoT) devices and distributed sensing infrastructure has created sustained demand for real-time data processing at scale. Tens of billions of connected devices are currently deployed globally, with projections indicating continued exponential growth through the latter half of this decade [1]. These devices collectively produce data at rates that strain existing processing infrastructure—in velocity and latency sensitivity. A broad and growing class of applications, spanning industrial process control, autonomous vehicle coordination, real-time financial market analysis, smart grid fault detection, and remote patient monitoring, requires that data be ingested, processed, and acted upon within millisecond timescales.

In these domains, latency is not a quality-of-service consideration but an operational constraint: a manufacturing defect undetected in time causes product loss; a trading signal processed late costs capital; a critical patient event unresolved within seconds compounds into irreversible harm. Real-time analytics at scale is now an operational requirement across these domains.

Cloud computing has served as the dominant paradigm for large-scale stream analytics over the past decade. Distributed stream processing frameworks such as Apache Flink [2], Apache Spark Streaming [3], and Apache Kafka Streams [4] have demonstrated exceptional scalability, fault tolerance, and expressiveness, enabling organizations to process high-throughput data streams through complex operator pipelines. These systems, however, are architecturally anchored to centralized cloud infrastructure. Data must traverse wide-area networks to reach cloud datacenters before any computation can occur, introducing round-trip latencies that typically range from 50 to 200 milliseconds depending on geographic proximity and network conditions [5]. While such latencies are acceptable for batch analytics and non-time-critical workloads, they cannot satisfy the sub-10-millisecond response requirements of latency-sensitive real-time applications [6]. The physical constraints of wide-area networking impose a hard ceiling on what cloud-only architectures can achieve, regardless of the computational power provisioned within the datacenter.

Edge computing addresses this limitation by relocating computation to the periphery of the network [7], proximate to the sources of data generation. By processing data at or near the point of origin—on gateway devices, on-premises servers, or base stations in Mobile Edge Computing (MEC) de-

ployments, edge architectures reduce network traversal distance and achieve latency profiles that cloud computing cannot match [6]. The ETSI Multi-access Edge Computing specification and the global rollout of 5G networks have substantially advanced the practical viability of edge analytics, providing standardized deployment models and low-latency connectivity that make edge processing increasingly tractable at industrial scale [8]. Surveys of deployed MEC systems have demonstrated that edge-local processing reduces end-to-end response latencies by one to two orders of magnitude compared to cloud-routed alternatives, with production measurements reporting sub-10 ms edge processing latencies against cloud-routed baselines exceeding 100 ms in representative IoT and industrial deployments [9] [10]. Despite these advantages, edge nodes remain resource-constrained: they operate under strict limits on available memory, processing capacity, and storage. Executing computationally intensive workloads at the edge frequently exceeds the resource envelope of available hardware. Stateful window aggregations, complex event pattern detection, and ML inference over large models demand memory and compute that edge nodes cannot sustain, leading to degraded throughput or system failure.

The complementary limitations of cloud-only and edge-only architectures have motivated growing interest in hybrid edge-cloud stream processing, wherein analytic workloads are distributed across both tiers according to their computational and latency requirements. Wang et al. [11] proposed an edge-cloud integrated framework for flexible stream analytics, demonstrating that hybrid deployment reduces inference latency while sustaining competitive throughput. Nazir [12] reported improvements in latency, privacy,

and horizontal scalability over cloud-only baselines through a hybrid cloud-edge framework. George [5] identified integration complexity and cross-tier consistency as the principal engineering challenges in hybrid multi-cloud deployments for real-time streaming. Pallikonda et al. [13] demonstrated measurable gains in energy efficiency and processing latency for real-time IoT workloads in hybrid edge-cloud deployments. Alsurdeh et al. [14] addressed workflow scheduling in edge-cloud systems through heuristics that exploit heterogeneous compute capabilities across tiers.

These contributions collectively establish the technical feasibility of hybrid stream processing architectures. However, a critical limitation persists: the workload offloading decision. determining which stream operators to execute at the edge, which to execute in the cloud, and when to migrate operators between tiers as conditions change. is not treated as a principled, adaptive, and latency-aware architectural component in any of this prior work. Offloading decisions are governed by static configuration parameters set at deployment time, by application-specific heuristics, or by coarse-grained rules that treat all operators uniformly regardless of their computational profiles, accumulated state size, or latency sensitivity [11, 12, 14].

This static approach is ill-suited to the dynamic reality of production deployments, where edge resource availability fluctuates [15] under workload contention, network round-trip times vary with mobility and interference, and stream ingest rates exhibit pronounced temporal irregularity. A placement policy fixed at deployment time will systematically underperform: it overloads edge nodes during demand spikes while cloud resources sit idle, and routes latency-critical operators to the cloud during periods when edge

capacity is available. The result is a class of hybrid architectures that are structurally hybrid in design but operationally rigid in behavior; structurally incapable of delivering the consistent low-latency guarantees that motivate their adoption.

This paper addresses this gap by presenting **HybridStream**, a stream processing framework designed for low-latency real-time analytics in dynamic, resource-heterogeneous deployment environments. The distinguishing innovation of HybridStream is its **Adaptive Offloading Decision Engine (AODE)**—a runtime architectural component that continuously monitors edge resource utilization, inter-tier network round-trip time (RTT), per-operator computational complexity, and stream ingestion rate trends, subject to user-defined latency service-level objectives (SLOs), to make fine-grained, dynamic decisions about operator placement. Operators are assigned to edge or cloud tiers based on a multi-factor scoring model that explicitly balances processing latency, resource consumption, and data transfer cost. When runtime conditions change, HybridStream transparently migrates operators between tiers without interrupting stream continuity or sacrificing state consistency. The edge tier is served by a purpose-built lightweight stream runtime optimized for resource-constrained environments, while the cloud tier integrates with existing Apache Flink deployments to leverage mature fault-tolerance and elastic scalability infrastructure. HybridStream thus combines the latency advantages of edge-proximate processing with the computational capacity of cloud infrastructure, governed by a runtime placement engine that continuously navigates between these two operational modes in response to measured system state.

The principal contributions of this work are as follows:

- **HybridStream Framework Architecture:** A design and implementation of a hybrid edge-cloud stream processing framework that supports dynamic, fine-grained operator placement across edge and cloud tiers, with transparent state migration and continuous stream execution.

- **Adaptive Offloading Decision Engine (AODE):** A latency-aware decision engine employing a multi-factor scoring model to determine optimal operator placement based on real-time monitoring of resource utilization, network RTT, operator complexity, and SLO constraints, with support for continuous recalibration at runtime without stream interruption.

- **Lightweight Edge Stream Runtime:** A purpose-built Python-based stream processing runtime for resource-constrained edge nodes that eliminates the Java Virtual Machine (JVM)-level cluster management infrastructure of Apache Flink, specifically the TaskManager and JobManager processes, reducing the idle memory footprint from approximately 1.2 GB (minimal Flink deployment) to approximately 55 MB (HEA baseline), a reduction of more than 95%. CPU-bound operators execute in isolated worker processes to bypass CPython's Global Interpreter Lock (GIL); I/O-bound operators run directly in the asyncio event loop. Quantified comparisons are reported in Section 7.

- **Empirical Evaluation:** An experimental evaluation of HybridStream against two baselines—a cloud-only Apache Flink deployment and a

statically partitioned hybrid configuration, across three representative workload scenarios under varying network and load conditions. Results demonstrate statistically significant reductions in end-to-end 95th-percentile (p95) processing latency, sustained SLO compliance under load spikes that cause baseline violations, and peak throughput within 6% of the statically partitioned hybrid baseline (B2), confirming that continuous adaptivity does not materially reduce processing capacity.

- **Reference Implementation:** A fully functional prototype of HybridStream, made publicly available to enable reproducibility of experimental results and to facilitate adoption and extension by the research community.

The remainder of this paper is organized as follows. Section 2 surveys related work across four thematic areas. centralized stream processing frameworks, edge and fog computing paradigm [16]s, hybrid edge-cloud architectures, and workload offloading strategies. positioning HybridStream within the existing research landscape. Section 3 formally defines the problem and motivates the design requirements of HybridStream through representative deployment scenarios. Section 4 presents the system architecture in detail, including the AODE scoring model and the operator migration protocol. Section 5 describes the implementation of the lightweight edge runtime and the cloud integration layer. Section 6 presents the experimental evaluation methodology, including workload definitions, baseline configurations, and performance metrics. Section 7 reports and analyzes experimental results. Section 8 concludes with a summary of contributions and outlines directions for future work.

2. Related Work

This section surveys the body of research most directly relevant to HybridStream, organized into four thematic areas: centralized stream processing frameworks, edge and fog computing paradigms, hybrid edge-cloud architectures, and workload offloading strategies. The section concludes by explicitly positioning HybridStream relative to the identified limitations in the existing literature.

2.1. Centralized Stream Processing Frameworks

The foundation of modern large-scale stream analytics was established by a generation of distributed processing frameworks designed for centralized cloud deployment. Apache Storm, introduced at Twitter by Toshniwal et al. [17]; resource-aware scheduling extensions such as T-Storm [18] and R-Storm [19] subsequently demonstrated that topology-aware placement improves throughput by 30–80% over default schedulers, demonstrated that continuous, real-time computation over unbounded data streams could be achieved in a distributed, fault-tolerant manner, and established the operator-graph model, wherein data flows through a directed acyclic graph (DAG) of processing nodes, that remains the dominant abstraction in stream processing to this day. Its successor, Apache Heron [20], addressed significant scalability and operational limitations of Storm by introducing a process-level isolation model and improved backpressure mechanisms, while maintaining API compatibility with existing Storm topologies.

Apache Spark Streaming [3] extended the batch-oriented Spark framework to support near-real-time processing through a micro-batch execu-

tion model, wherein continuous streams are discretized into small bounded batches. While this approach simplified consistency semantics and facilitated integration with Spark's rich analytical ecosystem, its micro-batch nature introduced an inherent latency floor proportional to the batch interval, typically between 500 milliseconds and several seconds, rendering it unsuitable for applications with strict sub-second latency requirements. Apache Flink [2] addressed this limitation by adopting a true record-at-a-time streaming model with native support for event-time processing, stateful computation, and exactly-once semantics through its distributed snapshot mechanism, establishing itself as the current state of the art for low-latency cloud stream processing. Resource-aware scheduling extensions such as T-Storm [18] and R-Storm [21] have addressed operator placement within single-cluster deployments by incorporating network topology and resource heterogeneity into scheduling decisions, but remain confined to centralized infrastructure. Apache Kafka Streams [4], while more limited in scope, provided a compelling lightweight processing option tightly coupled to the Kafka messaging infrastructure, well-suited for stateless and lightly stateful transformations at high throughput. A taxonomy and comparative evaluation of these systems is provided by Fragkoulis et al. [22], who identify operator placement and resource management as the most critical open challenges in the field. Hirzel et al. [23] provide a complementary optimization catalog for stream processing systems, establishing a taxonomy of placement and scheduling strategies that informs the design choices in this work. The foundational work on network-aware operator placement by Pietzuch et al. [24] established that placement-aware scheduling can reduce end-to-end latency by

50% or more in distributed stream processing systems. Heinze et al. [25] addressed elastic operator scaling in centralized stream processing, demonstrating latency-aware scale-out policies that adapt parallelism in response to load changes. Kalavri et al. [26] identified three fundamental steps for optimizing distributed data processing: profiling, placement, and routing, providing a systematic framework that informs HybridStream's multi-factor approach. The Dhalion framework [27] introduced self-regulating policies for Heron that detect and resolve bottlenecks automatically, establishing a precedent for runtime-adaptive stream processing that HybridStream extends to the cross-tier placement domain.

Despite their considerable contributions, all of these frameworks share a foundational architectural assumption: computation is performed within a centralized cluster, and data must travel to that cluster before any processing can occur. This premise is unproblematic within a data center environment but becomes a disqualifying constraint in latency-sensitive deployments where data originates at geographically distributed edge locations—the round-trip cost of reaching a centralized processor is incompatible with real-time response requirements.

2.2. Edge Computing and Fog Paradigms

The concept of fog computing, introduced by Bonomi et al. [28] in the context of IoT infrastructure, proposed a distributed computing architecture that extends cloud services to the network edge, enabling data processing, storage, and application services to be deployed in close proximity to end devices. This foundational work identified latency reduction, bandwidth conservation, and location awareness as the defining motivations for

edge-proximate computation—motivations that remain equally relevant as data volumes and device proliferation have continued to grow. Shi et al. [29] subsequently articulated a vision for edge computing as an infrastructure paradigm, characterizing the technical challenges of periphery deployment, including hardware heterogeneity, intermittent connectivity, security exposure, and the absence of standardized programming models, and establishing a research agenda that has shaped the field for nearly a decade.

Mobile Edge Computing (MEC), standardized by ETSI [8], formalized the deployment of cloud-like compute resources within mobile network infrastructure at base stations and radio access nodes. With 5G New Radio providing low-latency backhaul, single-digit-millisecond edge response times have become achievable in practice, validating MEC as the dominant deployment model for latency-sensitive analytics in cellular environments. The broader IoT-edge-cloud computing continuum. spanning the full spectrum from constrained end devices through edge and fog nodes to centralized cloud datacenters. has been surveyed by Kuchuk and Malokhvii [30], who identify stream processing at the edge as one of the most consequential open research frontiers within this landscape.

Across this literature, a consistent tension emerges: edge nodes deliver compelling latency advantages, but their constrained compute, memory, and storage capacities impose hard limits on the complexity of workloads they can sustain in isolation. Stateful aggregations, multi-stream joins, and inference over large models routinely exceed what edge hardware can absorb; making the case not for edge-only or cloud-only processing, but for a principled, adaptive distribution of work across both tiers.

2.3. Hybrid Edge-Cloud Stream Processing Architectures

Addressing the limitations of both pure-edge and pure-cloud execution, a body of work has explored hybrid architectures in which analytic workloads are partitioned across tiers. Ghosh and Simmhan [31] proposed one of the earliest tier-aware stream analytics frameworks, demonstrating that partitioning event processing graphs across edge and cloud nodes based on event dependency structure can reduce end-to-end latency while preserving correctness guarantees. Their work established the theoretical basis for cross-tier operator scheduling but treated placement as a one-time, deployment-time decision with no mechanism for runtime revision.

Wang et al. [11] built on this direction with an edge-cloud integrated framework specifically designed for flexible stream analytics, supporting configurable partitioning of analytic pipelines across edge inference nodes and cloud aggregation backends. Their evaluation demonstrated statistically significant latency reductions for inference-heavy workloads relative to cloud-only deployment; however, the partitioning policy was fixed at deployment time and remained unresponsive to changes in network conditions or edge resource availability during execution. NebulaStream [32] represents the most recent effort toward adaptive edge stream processing, providing a unified runtime that spans IoT devices, edge nodes, and cloud backends with support for query optimization across heterogeneous hardware. While NebulaStream addresses heterogeneity-aware query placement, its placement decisions are made at query submission time and do not recalibrate at runtime in response to changing resource conditions or SLO violations. Cardellini et al. [33] formulated the distributed stream processing operator placement problem as

an integer linear program, proving its NP-hardness and proposing heuristic solvers that achieve near-optimal placements under static workload assumptions. Their subsequent work [34] extended this to decentralized adaptation, though without cross-tier state migration support. Zhou et al. [35] approached the edge-cloud division problem from an edge intelligence perspective, examining how inference workloads and data pipelines can be partitioned across edge and cloud tiers through adaptive model compression, quantization, and progressive data transmission. Their framework adapts at runtime; but in the volume and fidelity of data transmitted between tiers, not in the computational topology itself, which remains fixed at deployment time.

Nazir [12] and George [5] each examined hybrid cloud-edge frameworks from architectural and systems perspectives, documenting improvements in latency, bandwidth utilization, and scalability over cloud-only baselines. Both studies relied on configuration-time workload partitioning and did not address the problem of runtime operator reallocation. Alsurdeh et al. [14], working in the related domain of scientific workflow scheduling, proposed heuristic-based algorithms for edge-cloud job placement that account for heterogeneous processing capacities and inter-tier communication costs. While their scheduling analysis provided useful insights, scientific workflows differ from continuous stream processing: workflow tasks are discrete and stateless, whereas stream operators are persistent, maintain internal state, and must preserve ordering and exactly-once semantics across any tier boundary. Pallikonda et al. [13] demonstrated energy efficiency and latency gains for IoT telemetry processing in hybrid deployment, with a focus on reducing commu-

nication overhead; but without support for general-purpose stream operator graphs or runtime reallocation of operators in response to changing resource conditions.

The consistent limitation across this body of work is the treatment of tier assignment as an engineering-time decision rather than a runtime concern. No existing hybrid stream processing framework exposes workload placement as an adaptive, continuously recalibrated function of live system state.

2.4. Workload Offloading and Task Scheduling

The task offloading literature, rooted in mobile edge computing research, frames the placement decision as an optimization problem over competing objectives: minimizing task completion latency, reducing device energy consumption, and maximizing resource utilization across available compute tiers. Mao et al. [36] provided a survey of dynamic computation offloading in MEC environments, cataloguing the key decision factors—task delay, energy expenditure, bandwidth utilization, and queue depth, and the optimization techniques applied to balance them. Their analysis established offloading as an inherently dynamic problem, one that must respond continuously to runtime changes in network quality and resource availability, a finding with direct implications for the stream processing setting.

Reinforcement learning has emerged as the dominant approach for adaptive offloading under uncertainty. Taheri-abed et al. [37] conducted a systematic review of ML-based computation offloading in edge and fog environments, finding that RL approaches consistently outperform both static and heuristic methods in environments characterized by irregular task arrival patterns and variable network conditions. Allaoui et al. [38] demon-

strated that RL-based offloading for IoT applications in fog computing can achieve latency reductions of 30–50% over heuristic baselines across dynamic workload scenarios. Ali et al. [39] evaluated deep reinforcement learning algorithms for joint task offloading and resource allocation, reporting strong performance but also surfacing a critical practical constraint: the inference overhead of DRL agents is itself non-trivial on resource-constrained edge hardware, motivating lighter-weight decision mechanisms that can operate at runtime without excessive compute cost.

In the stream processing domain specifically, Cardellini et al. [33] formulated the optimal operator placement problem for distributed stream processing and proved its NP-hardness, establishing the theoretical foundation upon which subsequent heuristic approaches build. Their follow-up work [34] extended this to decentralized self-adaptation for elastic data stream processing, achieving runtime adaptation through local operator decisions rather than centralized coordination. Nardelli et al. [40] further refined the placement formulation with efficient heuristics for heterogeneous infrastructures. Heinze et al. [25] addressed elastic auto-scaling for stream processing, demonstrating that proactive scaling based on load prediction outperforms reactive threshold-based approaches. Kalavri et al. [26] showed that three scaling decisions (scale-up, scale-down, and scale-out) suffice for effective stream processing adaptation. Floratou et al. [27] introduced Dhalion, a self-regulating stream processing system for Heron that detects and resolves performance issues through automated diagnosis and action policies. Pietzuch et al. [41] proposed network-aware operator placement that accounts for inter-node latency when distributing operators across a wide-area overlay network. Akhter

et al. [42] explored stateful functions as a service within the Flink ecosystem, enabling fine-grained stateful computation that is relevant to cross-tier execution models.

Despite the depth of the offloading literature, a structural mismatch limits its applicability to stream processing: existing frameworks are designed around discrete, independent, stateless task units. compute jobs with a defined start, end, and no persistent state. Stream processing operators are structurally different: they run continuously, accumulate state across time windows, and exist within interdependent operator graphs where the placement of one operator constrains the feasible placements of its neighbors. Migrating a stateful stream operator between tiers requires serializing and transferring its state, re-establishing its position within the data flow, and maintaining processing semantics across the transition. none of which is addressed by the task offloading literature. HybridStream's AODE addresses this gap.

2.5. Positioning HybridStream

Table 1 summarizes the relationship between HybridStream and the most directly related prior systems across five dimensions: offloading granularity, runtime adaptivity, support for stateful operators, state migration capability, and continuous stream execution.

As Table 1 illustrates, HybridStream is the only system in this comparison to satisfy all five dimensions simultaneously. Cardellini et al. [34] come closest, achieving operator-level adaptive placement with stateful support, but their decentralized approach does not include cross-tier state migration; operators are re-instantiated rather than migrated, losing accumulated

Table 1: Comparison of hybrid edge-cloud systems across five architectural dimensions.

System	Offloading	Adaptive?	Stateful	Migration	Continuous
Wang et al. [11]	Pipeline stage	No	Yes	No	Yes
Alsurdeh et al. [14]	Workflow	No	No	No	No
Ghosh & Simmhan [31]	Event graph	No	Partial	No	Partial
Zhou et al. [35]	Adaptive data	Partial	Partial	No	Yes
Allaoui et al. [38]	Task-level	Yes (RL)	No	No	No
NebulaStream [32]	Query-level	Partial	Partial	No	Yes
Cardellini et al. [34]	Operator	Yes	Yes	No	Yes
NebulaStream [32]	Query-level	Partial	Yes	No	Yes
Cardellini et al. [33]	Operator	Partial	Yes	No	Yes
HybridStream	**Operator**	**Yes (AODE)**	**Yes**	**Yes**	**Yes**

state. NebulaStream [32] supports adaptive query optimization across heterogeneous devices but operates at query granularity rather than individual operator granularity, and does not provide a state migration protocol. Prior hybrid architectures either lack runtime adaptivity, do not support stateful operator migration, or were not designed for continuous stream workloads. Prior offloading systems are adaptive but operate on discrete, stateless tasks and cannot be applied to stream operator graphs without fundamental redesign. HybridStream unifies these two lines of work into a single, coherent framework; and in doing so, addresses a gap that has persisted across the hybrid stream processing and edge offloading literatures. Section 3 formalizes this gap through a precise problem statement and motivating deployment scenarios, from which the design requirements of HybridStream are derived.

3. Problem Statement & Motivation

This section formalizes the problem that HybridStream is designed to solve. We first introduce the system model, then state the placement optimization problem, present three representative deployment scenarios that illustrate the practical consequences of static placement policies, and finally derive the design requirements that guide the architecture of HybridStream.

3.1. System Model

We consider a two-tier deployment environment consisting of a set of edge nodes $\mathbf{E} = \{\mathrm{e}_1, \mathrm{e}_2, \ldots, \mathrm{e}_n\}$ and a cloud cluster $\mathbf{C}$. Edge nodes are resource-constrained compute units deployed proximate to data sources; the cloud cluster provides elastic, high-capacity computation and persistent storage. A stream processing application is modeled as a directed acyclic graph $\mathbf{G} = (\mathbf{O}, \mathbf{D})$, where $\mathbf{O} = \{\mathrm{o}_1, \mathrm{o}_2, \ldots, \mathrm{o}_k\}$ is a finite set of stream operators and $\mathbf{D} \subseteq \mathbf{O} \times \mathbf{O}$ represents the data flow dependencies between them. We adopt the DAG model standard in the stream processing literature [2, 17, 20]; extensions to cyclic topologies are left as future work. Each operator $o_i \in \mathbf{O}$ is characterized by the following attributes:

- **Computational cost** $c_i(\mathrm{x})$: the CPU resources required to process x records per second
- **State size** s_i: the memory consumed by the operator's accumulated state at steady throughput
- **Latency sensitivity** $\lambda_i \in \{\text{critical, standard, batch}\}$: the operator's tolerance for processing delay

- **Output rate** r_i(x): the number of records emitted per second given an input rate of x

Each edge node $e_j \in \mathbf{E}$ has a compute capacity CPU_j and a memory capacity MEM_j. The network round-trip time between edge node e_j and the cloud cluster at time t is denoted τ_j(t), which varies due to network congestion, interference, and mobility. The data transfer cost for streaming at rate r from e_j to the cloud at time t is denoted β_j(r, t), capturing bandwidth consumption and egress cost. We assume edge nodes operate reliably within each evaluation period; handling edge node failures is orthogonal to placement optimization and is treated as a system-level concern addressed by the underlying infrastructure.

3.2. Problem Formulation

A **placement assignment** is a function π: $\mathbf{O} \to \mathbf{E} \cup \{\mathbf{C}\}$ that maps each stream operator to a compute tier. A placement π is *feasible* if and only if, for every edge node $e_j \in \mathbf{E}$, the aggregate computational and memory demands of all operators assigned to e_j do not exceed its physical capacity:

$$\forall e_j \in \mathbf{E} : \sum_{\{o_i : \pi(o_i) = e_j\}} c_i(x) \leq \mathrm{CPU}_j \quad \text{and} \quad \sum_{\{o_i : \pi(o_i) = e_j\}} s_i \leq \mathrm{MEM}_j \tag{1}$$

The **end-to-end latency** of operator o_i under placement π is:

$$L(o_i, \pi) = p_i(\pi(o_i)) + \sum_{\{o_j \to o_i\}} \tau(\pi(o_j), \pi(o_i)) \tag{2}$$

where $p_i(\pi(o_i))$ is the processing latency at the assigned tier, and $\tau(\pi(o_j), \pi(o_i))$ is the inter-tier communication delay for each upstream dependency o_j that feeds into o_i (zero when both operators are on the same tier).

The **placement optimization problem** is then:

$$\begin{aligned} \underset{\pi}{\text{minimize}} \quad & \sum_i \nu_i \cdot L(o_i, \pi) \\ \text{subject to:} \quad & \text{feasibility constraints (Eq. 1)} \\ & L(o_i, \pi) \leq \text{SLO}_i \;\; \forall\, o_i \text{ with } \lambda_i = \text{critical} \end{aligned} \tag{3}$$

where ν_i is the application-assigned importance weight of operator o_i and SLO_i is its user-defined latency bound. (Note: ν_i is distinct from the AODE scoring weights $\mathbf{w} = (\text{w}_1,\ \text{w}_2,\ \text{w}_3,\ \text{w}_4)$ introduced in Section 4.2.2, which govern the relative contribution of each scoring factor; ν_i governs how much each operator's latency contributes to the global objective.)

This problem is NP-hard in the general case, by polynomial reduction from the bin-packing problem [43]: each operator corresponds to an item with size $c_i(\text{x})$ (CPU) and s_i (memory), each edge node to a bin with capacity CPU_j and MEM_j, and the feasibility constraints directly encode the bin-packing constraint, with the latency minimization objective adding additional complexity that does not relax the underlying hardness. Moreover, the placement must be solved *continuously* at runtime as $\tau_j(\text{t})$, $\beta_j(\text{r, t})$, and available capacities change; making exact optimal solutions computationally intractable at the frequency required for responsive adaptation. HybridStream's AODE addresses this with a polynomial-time multi-factor scoring heuristic, whose effectiveness relative to optimal placement is evaluated em-

pirically in Section 6.

3.3. Motivating Scenarios

The following three scenarios, drawn from representative deployment domains, illustrate the practical consequences of static placement policies and motivate the adaptive design of HybridStream.

Scenario 1: Industrial IoT Defect Detection. A manufacturing facility deploys 200 vibration and thermal sensors on a high-speed production line. Raw sensor streams are ingested at 10 kHz per sensor, and a classification operator must emit a go/no-go decision within 5 milliseconds to trigger line shutdown before a faulty component progresses to the next assembly stage. Under a cloud-only Apache Flink deployment, round-trip latency to the datacenter is 80ms. 16× the SLO. A statically configured edge deployment maintains compliance during normal operation but degrades under sustained peak-shift load as edge CPU becomes saturated, causing processing backlog and cascading SLO violations. To address this failure mode, HybridStream is designed to retain the classification operator on the edge and migrate the upstream feature aggregation operator to the cloud when edge utilization crosses a configurable threshold. sustaining SLO compliance across variable load conditions without operator intervention.

Scenario 2: Smart City Traffic Analytics. A municipal traffic management system ingests camera feeds from 500 intersections. Per-intersection vehicle detection is latency-critical. the system must respond to incidents within 2 seconds to redirect traffic before queues form. Cross-city flow aggregation and long-horizon pattern modeling are compute-intensive but latency-tolerant. A static hybrid configuration performs well under average condi-

tions but degrades during peak hours when detection operators compete with aggregation for edge memory, causing backpressure that increases processing latency and ultimately violates the 2-second detection SLO. Rather than reacting after violation, the AODE is designed to anticipate memory pressure through continuous resource trend monitoring and trigger preemptive migration of aggregation operators to the cloud before saturation is reached. a *proactive* failure mode distinct from the reactive CPU-bound response of Scenario 1.

Scenario 3: Financial Market Stream Processing. A trading firm processes tick-by-tick market data from 30 exchanges at a combined rate of 500,000 events per second. Pre-trade risk checks and anomaly detection carry sub-millisecond SLOs; compliance logging and statistical aggregation are data-intensive but latency-tolerant. A cloud-only deployment introduces 60ms latency—acceptable for logging but catastrophic for risk checks. A fixed edge deployment handles normal volumes but collapses under news-driven event spikes that can triple ingest rate within seconds. For this third failure mode, bursty, unpredictable throughput, the AODE is designed to monitor ingest-rate trends and proactively offload latency-tolerant operators to the cloud *before* a spike saturates edge capacity, faster than any human operator could intervene.

Together, these three scenarios establish three distinct failure modes, sustained CPU saturation, concurrent memory contention, and unpredictable throughput spiking, each demanding qualitatively different AODE behaviors: reactive operator migration, proactive preemption, and anticipatory trend-based offloading, respectively. These behaviors are formalized in the

AODE scoring model presented in Section 4.

3.4. Design Requirements

The problem formulation and motivating scenarios yield six design requirements that HybridStream must satisfy:

- **R1. Operator-level granularity:** Placement decisions must be made at the level of individual operators, not pipeline stages or application instances, to exploit the heterogeneous latency and resource profiles of different operators within a single application.

- **R2. Runtime adaptivity:** Placement must be continuously reassessed and updated in response to measured changes in edge resource utilization, inter-tier network RTT, and stream ingestion rates, without requiring operator intervention or application restart.

- **R3. Stateful migration:** Operator migration between tiers must preserve accumulated operator state, maintaining exactly-once processing semantics and preventing data loss or duplication during tier transitions.

- **R4. SLO-aware prioritization:** Under resource contention, placement decisions must unconditionally prioritize SLO compliance for latency-critical operators over resource efficiency for non-critical operators. SLO satisfaction is a hard constraint, not a preference.

- **R5. Lightweight decision overhead:** The placement decision mechanism must impose overhead that is negligible relative to the application's own resource consumption on edge nodes, ensuring that the

AODE does not meaningfully compete with application operators for the resource budget it manages.

- **R6. Cloud framework compatibility:** The cloud tier must integrate with existing Apache Flink deployments without requiring modification to application operator code, enabling adoption without forcing migration of existing stream processing infrastructure.

These six requirements collectively define the design space of HybridStream. Requirements R1, R2, and R4 are addressed by the AODE design presented in Section 4.2; R3 by the operator migration protocol in Section 4.3; R5 by the polynomial-time recalibration algorithm and asynchronous telemetry collection described in Section 4.2.3; and R6 by the cloud integration layer in Section 5.2. The lightweight edge runtime (Section 5.1) is an enabler of the overall system: it provides the execution substrate through which R1–R4 can be realized on resource-constrained hardware.

4. Proposed Architecture

This section presents the architecture of HybridStream in three parts. Section 4.1 describes the four principal system components and their interactions at a structural level. Section 4.2 details the Adaptive Offloading Decision Engine (AODE), the runtime intelligence that drives placement decisions, decomposed into its telemetry collection subsystem (Section 4.2.1), multi-factor operator scoring model (Section 4.2.2), and placement recalibration algorithm (Section 4.2.3). Section 4.3 describes the stateful operator migration protocol through which AODE placement directives are enacted

without interrupting stream execution or violating exactly-once semantics. The mapping of each subsection to the design requirements established in Section 3.4 is summarized in Table 2.

Table 2: Design requirement coverage by architecture subsection.

Requirement	Description	Addressed in
R1	Per-operator placement decisions	§4.2.2, §4.2.3
R2	Continuous recalibration	§4.2.1, §4.2.3
R3	Exactly-once state transfer	§4.3
R4	SLO prioritization	§4.2.2
R5	Lightweight overhead	§4.2.3
R6	Cloud framework compatibility	§4.1

4.1. System Overview

HybridStream is structured as four loosely coupled components that interact through well-defined interfaces: the **Stream Ingestion Layer**, the **Edge Processing Tier**, the **Cloud Processing Tier**, and the **Adaptive Offloading Decision Engine (AODE)**. Figure 1 depicts the high-level architecture, showing primary data paths (solid arrows) and AODE control channels (dashed arrows).

Stream Ingestion Layer. All event streams, from IoT sensors, instrumented applications, financial data feeds, or any Kafka-compatible producer, enter HybridStream through an Apache Kafka cluster. Kafka serves three roles simultaneously: it decouples data producers from the processing tier (allowing producers to remain unaware of operator placement), it provides

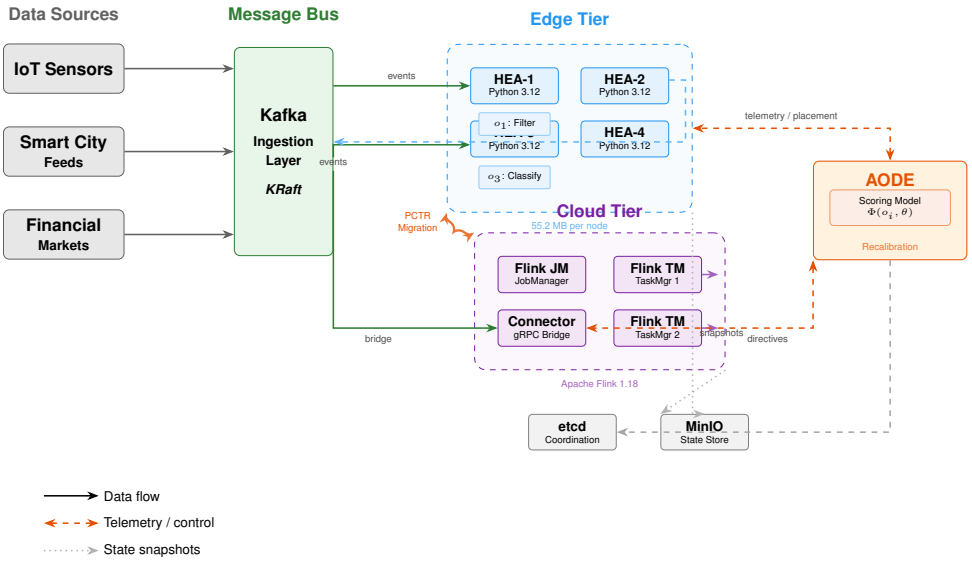

Figure 1: HybridStream system architecture. Solid arrows: data flow. Dashed arrows: AODE telemetry and control. The ingestion layer (Kafka) decouples producers from both processing tiers; inter-tier data exchange is routed through dedicated Kafka bridge topics.

ordered, durable, and replay-capable record storage that prevents data loss during operator migrations, and it acts as the inter-tier transport substrate when operators on different tiers must exchange data. Kafka topics are partitioned by a source-domain key (e.g., device zone, market identifier) so that related partitions are assigned to the geographically nearest edge node's consumer group, minimizing unnecessary wide-area data movement at ingest time.

Edge Processing Tier. Each edge node e_j runs a **HybridStream Edge Agent** (HEA), a purpose-built, resource-efficient stream runtime described in full in Section 5.1. The HEA is responsible for three functions: executing the subset of the application's operator DAG currently assigned to e_j by the AODE; maintaining accumulated operator state in a local in-memory state store backed by periodic write-ahead checkpoints to durable local storage; and exposing a gRPC management API through which the AODE polls resource metrics and delivers placement directives. When an operator on e_j produces output consumed by a downstream operator placed on the cloud tier, the HEA serializes those records and publishes them to a dedicated Kafka bridge topic, keeping inter-tier data transfer asynchronous, buffered, and decoupled from the producing operator's execution path. This arrangement ensures that transient inter-tier latency spikes do not propagate backpressure into the edge execution engine.

Cloud Processing Tier. The cloud tier hosts an Apache Flink cluster augmented with a **HybridStream Flink Connector**—a thin adapter layer that integrates the Flink JobManager with the AODE control plane without requiring modifications to application operator code (R6). Opera-

tors assigned to the cloud tier execute as standard Flink operators within a Flink JobGraph; the connector translates incoming AODE placement directives into Flink savepoint-and-rescale operations, leveraging Flink's native fault-tolerance infrastructure. Cloud-hosted operators consume their inputs from Kafka bridge topics published by edge-hosted upstreams and write their outputs to Kafka sinks for further processing or external delivery. Because the Flink integration operates entirely through the public Flink API, organizations with existing Flink deployments can adopt HybridStream's cloud tier with no changes to production operator implementations.

Adaptive Offloading Decision Engine (AODE). The AODE is the coordinating intelligence of HybridStream, deployed as a standalone high-availability service pair, a primary instance and a hot standby, each reachable from all HEAs and the Flink connector via gRPC (gRPC Remote Procedure Call, a high-performance open-source RPC framework developed by Google [44]). The AODE does not participate in the data plane: it neither produces nor consumes event records. It operates exclusively on the control plane, executing a continuous recalibration loop that (i) collects system telemetry, (ii) scores all candidate operator placements, (iii) identifies beneficial reallocation opportunities, and (iv) issues migration directives. Crucially, the AODE holds a global view of placement state, operator DAG structure, resource availability, and network conditions, a view that no individual HEA or Flink instance possesses. Separating the AODE from the execution tier ensures that a transient AODE unavailability does not disrupt active stream processing: operators continue executing at their last-known placement until the AODE primary or standby resumes issuing directives. The AODE

state, current placement assignments, recent telemetry, and migration status, is persisted to a local replicated key-value store (etcd [45], a distributed, strongly consistent key-value store commonly used for distributed system coordination) so that standby promotion does not require re-bootstrapping from connected HEAs.

4.2. Adaptive Offloading Decision Engine

The AODE comprises three cooperating sub-components: the **Telemetry Collector**, which gathers runtime system state; the **Operator Scoring Model**, which evaluates placement alternatives; and the **Placement Recalibration Algorithm**, which translates scores into placement decisions and migration directives. The recalibration loop runs at a configurable interval (default: 5 seconds). At this frequency, the full loop completes in under 200ms on commodity hardware in the worst case, and is empirically measured at 18–36ms on the evaluation platform described in Section 6.1, imposing negligible overhead relative to operator execution (R5).

4.2.1. Telemetry Collector

At the start of each recalibration cycle, the Telemetry Collector dispatches parallel gRPC telemetry requests to all registered HEAs and to the Flink connector, collecting the following signals:

- **Edge CPU utilization** $u_j^{\text{cpu}}(t)$: the fraction of available CPU cycles consumed across all operators hosted on e_j at time t, sampled as a 5-second average reported by each HEA.
- **Edge memory utilization** $u_j^{\text{mem}}(t)$: the fraction of available heap

consumed by operator state stores on e_j, including serialized window buffers and accumulator structures.

- **Inter-tier round-trip time** τ_j(t): the exponentially weighted moving average (EWMA)-smoothed round-trip time between edge node e_j and the cloud tier's Kafka ingress endpoint, updated every 500ms by a dedicated lightweight probe thread in each HEA. The EWMA update rule is $\tau_j(\mathrm{t}) = (1 - \alpha) \cdot \tau_j(\mathrm{t}–1) + \alpha \cdot \mathrm{m}(\mathrm{t})$ **(Eq. 2)**, where m(t) is the latest RTT measurement and smoothing factor $\alpha = 0.2$ was selected to suppress sub-second network jitter while remaining responsive to sustained RTT shifts over 2–3 seconds; this value was validated against network traces collected across the three evaluation deployments described in Section 6.

- **Stream ingest rate** $\rho(t)$: the aggregate event arrival rate across all Kafka partitions assigned to HybridStream, computed as a 30-second sliding window average by the Telemetry Collector over the Kafka consumer group lag metrics API, avoiding any per-HEA coordination.

- **Per-operator p95 latency** l_i(t): the 95th-percentile end-to-end processing latency of operator o_i over the most recent recalibration interval, reported by the HEA or Flink connector hosting o_i. This metric directly feeds the SLO compliance check in the scoring model (§4.2.2).

The Telemetry Collector retains a rolling 5-minute history of all collected metrics in an in-memory circular buffer. This history serves two purposes. First, it drives **ingest rate trend detection**, in which an ordinary least-squares (OLS) linear regression over the most recent 60 seconds of $\rho(t)$ sam-

ples yields a trend slope $\rho'(t)$ and its associated p-value, computed via the standard t-statistic for the OLS slope coefficient. In the Python implementation, this regression is computed using `scipy.stats.linregress` [46] at each recalibration cycle, completing in under 1ms (0.3ms measured on the evaluation platform; Section 5.2.1) for 60-second windows comprising 12 data points at the 5-second telemetry sampling interval. A statistically significant positive trend ($p < 0.05$) activates the scoring model's forward-looking adjustment described in §4.2.2, enabling the AODE to respond proactively to emerging load increases—the anticipatory offloading behavior motivated by Scenario 3. Second, the metric history enables the recalibration algorithm to distinguish sustained resource pressure (which warrants migration) from transient spikes (which do not), through the hysteresis mechanism described in §4.2.3.

Any HEA that fails to respond to its telemetry request within a 1-second deadline is marked as unreachable. All operators currently hosted on an unreachable HEA are immediately flagged for emergency cloud migration, and their corresponding Kafka bridge topics are promoted to active status to prevent consumer stalls on downstream operators.

4.2.2. Operator Scoring Model

The scoring model evaluates every (operator, tier) combination and assigns a scalar **placement score** $\Phi(o_i, \theta)$ that quantifies the expected quality of placing operator o_i on tier θ given current system state. Scores are non-negative; lower is better. An assignment is *feasible* if it satisfies the capacity constraints of Section 3.2 and *infeasible* otherwise; infeasible assignments receive a score of $+\infty$ and are never selected.

The placement score decomposes into four normalized factors:

$$\Phi(o_i, \theta) = w_1 \cdot \Phi_{\text{lat}}(o_i, \theta) + w_2 \cdot \Phi_{\text{res}}(o_i, \theta) + w_3 \cdot \Phi_{\text{net}}(o_i, \theta) + w_4 \cdot \Phi_{\text{slo}}(o_i, \theta) \quad (4)$$

where $\mathrm{w}_1 + \mathrm{w}_2 + \mathrm{w}_3 + \mathrm{w}_4 = 1$, $\mathrm{w}_1, \mathrm{w}_2, \mathrm{w}_3, \mathrm{w}_4 \geq 0$. All four component scores are dimensionless and lie in [0, 1] under normal operating conditions; the SLO urgency factor may exceed 1 by design, as explained below. The weight vector $\mathbf{w} = (\mathrm{w}_1, \mathrm{w}_2, \mathrm{w}_3, \mathrm{w}_4)$ is configured at the application level; three calibrated presets are provided: **latency-first** (0.55, 0.10, 0.20, 0.15) for latency-dominated workloads such as financial tick processing; **balanced** (0.30, 0.30, 0.20, 0.20) for general-purpose deployments; and **resource-efficient** (0.20, 0.50, 0.20, 0.10) for capacity-constrained edge deployments where preserving headroom for future operators takes priority.

Latency factor $\Phi_{\text{lat}}(o_i, \theta)$. This factor estimates the expected end-to-end processing latency for o_i at tier θ, expressed as a fraction of its SLO bound:

$$\Phi_{\text{lat}}(o_i, \theta) = \min\left(\frac{\hat{p}_i(\theta)}{\text{SLO}_i},\ 1\right) \quad (5)$$

where $\hat{p}_i(\theta)$ is the projected processing latency at tier θ. For an edge assignment $\theta = e_j$, the latency is modeled as a load-scaled estimate:

$$\hat{p}_i(e_j) = p_i^0 \cdot \left(1 + \kappa \cdot u_j^{\text{cpu}_{\text{eff}}}(t)\right) \quad (6)$$

Here p_i^0 is the operator's baseline per-record processing latency at zero additional load, maintained as a running minimum of $l_i(\mathrm{t})$ over the most recent 10-minute window, and $\kappa = 2.0$ is a CPU contention coefficient derived

empirically from profiling the three evaluation workloads at utilization levels ranging from 0% to 100% in 10% increments; the value 2.0 represents the median worst-case latency multiplier observed at 100% core utilization, consistent with queuing-theoretic predictions under an M/D/1 processor-sharing model [47]. The effective utilization $u_j^{\text{cpu}_{\text{eff}}}(t)$ accounts for the incremental load that would result from placing o_i on e_j: $u_j^{\text{cpu}_{\text{eff}}}(t) = u_j^{\text{cpu}}(t) + c_i(\rho(t))$ / CPU_j, where $c_i(\rho(t))$ is the estimated CPU demand of o_i at the current ingest rate, derived from the operator's empirical compute-per-record profile collected over prior recalibration cycles. For a cloud assignment $\theta = \mathbf{C}$, $\hat{p}_i(\mathbf{C}) = p_i^{\text{cloud}} + \tau_{\text{up}}(t) + \tau_{\text{down}}(t)$, where p_i^{cloud} is the operator's observed cloud processing latency and $\tau_{\text{up}}(t)$, $\tau_{\text{down}}(t)$ are the upstream and downstream inter-tier RTT contributions applicable only when data must cross tier boundaries to reach o_i or continue from it.

Resource pressure factor $\Phi_{\text{res}}(o_i, \theta)$. This factor penalizes edge assignments that would exhaust the physical resource envelope of the target node:

$$\Phi_{\text{res}}(o_i, \theta = e_j) = \max\left(\frac{u_j^{\text{cpu}}(t) \cdot \text{CPU}_j + c_i(\rho(t))}{\text{CPU}_j}, \frac{u_j^{\text{mem}}(t) \cdot \text{MEM}_j + s_i}{\text{MEM}_j}\right) \tag{7}$$

$$\Phi_{\text{res}}(o_i, \theta = \mathbf{C}) = 0$$

The formula evaluates the projected post-assignment utilization fraction for both CPU and memory, taking the higher of the two as the binding resource pressure signal. A value exceeding 1.0 indicates that the assignment would violate a hard capacity constraint; such assignments are immediately

classified as infeasible. The cloud tier receives a resource pressure score of zero, reflecting its elastic capacity model. Assigning a non-zero score even for sub-1.0 projected utilization ensures that the scoring model exerts graduated preference for placements that preserve edge headroom, not merely those that avoid outright overflow, supporting the proactive memory-pressure response identified in Scenario 2.

Network transfer factor $\Phi_{\mathbf{net}}(o_i, \theta)$. This factor captures the latency cost of cross-tier data movement incurred when o_i is placed on a different tier from one or more of its upstream producers:

$$\Phi_{\text{net}}(o_i, \theta) = \frac{1}{\Gamma} \sum_{\{o_j \to o_i : \pi'(o_j) \neq \theta\}} \tau_{\text{net}}(\pi'(o_j), \theta) \cdot r_j(\rho(t)) \tag{8}$$

where $\pi'(o_j)$ is the placement of upstream operator o_j as determined in the current recalibration round (in topological processing order, upstreams are scored before their dependents, so $\pi'(o_j)$ is already fixed when o_i is scored), $r_j(\rho(t))$ is the estimated output record rate of o_j at current ingest rate $\rho(t)$, $\tau_{\text{net}}(\pi'(o_j), \theta)$ is the inter-tier RTT on the path between the tiers of o_j and o_i (zero when same-tier, τ_j(t) when cross-tier), and Γ is a normalization constant set to the 99th-percentile of the quantity inside the sum observed over a 30-minute calibration run executed prior to production deployment, during which each edge node is subjected to a synthetic load sweep at full ingest rate. This calibration run is a one-time offline step that is fully automated by the HybridStream deployment toolchain. The product $\tau_{\text{net}} \cdot r_j(\rho(t))$ captures the latency-bandwidth exposure of each cross-tier dependency: τ_{net} represents the per-packet round-trip delay, while $r_j(\rho(t))$ represents the record throughput that must traverse that link, yielding a dimensionless cost pro-

portional to the number of records in flight across the tier boundary at any instant. This combined metric is more discriminating than RTT alone. two operators with equal RTT but different output rates impose substantially different cross-tier transfer costs. and directly encodes the empirical finding of Wang et al. [11] that cross-tier transfer overhead dominates end-to-end latency in hybrid deployments under degraded WAN conditions. An operator with many high-rate upstream dependencies therefore benefits strongly from co-location with those upstreams, as reflected by a lower Φ_{net} when assigned to the same tier.

SLO urgency factor $\Phi_{\text{slo}}(o_i, \theta)$. The preceding three factors drive the AODE toward low-latency, resource-efficient, and network-aware placements in aggregate. The SLO urgency factor provides a hard override: any placement projected to violate the operator's SLO receives a score penalty that dominates all other factors, ensuring that SLO compliance is treated as a constraint rather than a preference—satisfying R4:

$$\Phi_{\text{slo}}(o_i, \theta) = M \cdot \max\left(0,\ \frac{\hat{p}_i(\theta) + \sum_{\{o_j \to o_i : \pi'(o_j) \neq \theta\}} \tau_{\text{net}}(\pi'(o_j), \theta) - \text{SLO}_i}{\text{SLO}_i}\right) \tag{9}$$

where M = 10 is a scaling constant chosen so that a placement projecting even a 10% SLO violation inflates the total score by an amount equal to the entire [0,1] range of the other three factors combined, making it impossible for any combination of latency, resource, and network savings to justify an SLO-violating assignment unless all alternatives are also SLO-violating. For operators with λ_i = batch, for which no SLO is defined, the factor is set to zero, avoiding artificial score inflation for latency-indifferent operators.

Similarly, Φ_{lat} for batch operators uses a nominal SLO of 3,600 seconds (one hour), effectively normalizing their latency score to near-zero and deferring their placement entirely to the resource pressure and network transfer factors.

Trend-adjusted scoring. When the Telemetry Collector detects a statistically significant positive ingest rate trend, the scoring model substitutes the forward-projected ingest rate $\hat{\rho}(t+H) = \rho(t) + \mathrm{H} \cdot \rho'(t)$ **(Eq. 3)**—with lookahead horizon $\mathrm{H} = 30$ seconds, for the current rate $\rho(t)$ in all computations of $c_i(\cdot)$, $r_j(\cdot)$, and the resource pressure factor. This forward-looking substitution causes the model to evaluate placements against the anticipated near-term load state rather than the present snapshot, enabling the AODE to initiate preemptive migrations before resource constraints become binding, the anticipatory behavior that distinguishes HybridStream from reactive-only hybrid systems in the related work (Section 2.3). When no significant trend is detected, no substitution is made and the model operates on current measurements.

4.2.3. Placement Recalibration Algorithm

Algorithm 1 presents the full recalibration procedure executed by the AODE at each interval.

The algorithm proceeds in topological DAG order (line 3), evaluating each operator's placement alternatives in a pass where upstream assignments, already fixed earlier in the same pass, are taken as given when computing Φ_{net} for downstream operators. This ordering ensures that inter-tier transfer costs are computed against the placement decisions made in the current round rather than the prior round, producing self-consistent scores within a single recalibration cycle.

Algorithm 1 AODE Placement Recalibration

Require: $G = (O, D)$—operator DAG; π, current placement; T, telemetry; $\Delta_h = 0.15$

Ensure: π_{new}, updated placement; M, migration directives

```
π_new ← π; M ← ∅
order ← TopologicalSort(G)
for each o_i in order do
    θ* ← π(o_i); Φ* ← Φ(o_i, π(o_i); T, π_new)
    for each θ ∈ E ∪ {C} do
        if Feasible(o_i, θ, π_new, T) then
            s ← Φ(o_i, θ; T, π_new)
            if s < Φ* then
                Φ* ← s; θ* ← θ
            end if
        end if
    end for
    if θ* ≠ π(o_i) and Φ(o_i, π(o_i)) − Φ* > Δ_h then
        π_new(o_i) ← θ*; M ← M ∪ {(o_i, π(o_i), θ*)}
    end if
end for
PersistPlacement(π_new)
for each (o_i, src, dst) in TopologicalOrder(M) do
    Migrate(o_i, src, dst)
end for
return π_new
```

Hysteresis (line 14). A migration directive is issued only when the score improvement from moving o_i to best_θ exceeds the threshold $\Delta_h = 0.15$. This prevents placement oscillation driven by telemetry noise: small score fluctuations around a placement boundary, common when a node operates near its utilization threshold, will not repeatedly trigger migrations that consume migration overhead and interrupt downstream operators. The value $\Delta_h = 0.15$ was selected empirically by simulating recalibration over 48-hour telemetry traces from each evaluation scenario; it eliminates oscillation in all three workloads without delaying migrations that improve latency by more than 8% of SLO budget. The hysteresis approach is consistent with the findings of De Matteis and Mencagli [48], who demonstrated that foresight-based strategies with dampening reduce unnecessary reconfigurations in elastic stream processing. The threshold is configurable for deployments with higher noise floors or stricter responsiveness requirements.

Migration ordering (line 20). When multiple operators migrate in the same recalibration round, directives are issued in topological order; upstream operators first. This guarantees that a downstream operator's new upstream source is already established and forwarding live records before the downstream operator completes its own migration, preventing any consumer from being left with an inoperative upstream during the migration window.

Computational complexity. The recalibration loop runs in $O(|\mathbf{O}| \cdot (|\mathbf{E}| + 1))$ time in the scoring phase (lines 4–18), plus $O(|\mathbf{O}| \cdot \log |\mathbf{O}|)$ for topological sort. For the workloads evaluated in Section 6 (up to 30 unique operator types, 4 HEA edge nodes), the full scoring pass completes in 18–36ms on the AODE evaluation host, well within the 5-second recalibration

interval and satisfying requirement R5. The AODE scores operator types, not individual operator instances; for workloads with high operator instance parallelism (e.g., W2, Section 6.2), all instances of the same operator type are assigned collectively to a tier, with intra-tier instance distribution managed by each HEA's worker pool independently.

4.3. Stateful Operator Migration Protocol

When Algorithm 1 issues a migration directive (o_i, θ_{src}, θ_{dst}), the migration protocol must atomically transfer o_i's accumulated state, redirect its data flow, and establish it as an active processor on θ_{dst}; all without dropping or duplicating any in-flight records (R3). HybridStream implements a four-phase **pause-checkpoint-transfer-resume** (PCTR) protocol that achieves these properties by leveraging Kafka's ordered, durable storage as both a migration buffer and a consistency anchor.

Phase 1—Upstream Pause. The AODE sends a PAUSE directive to each of o_i's direct upstream operators (hosted on their respective HEAs or on the Flink connector). Upon receiving this directive, each upstream ceases forwarding output records to o_i's active input Kafka topic and begins writing all subsequent output for o_i to a dedicated per-migration **migration buffer** Kafka topic instead. The upstream operator continues processing its own input normally, it does not pause execution, but routes o_i-destined output exclusively to the migration buffer for the duration of the migration. Because the migration buffer is a Kafka topic, it provides durable, ordered, and unbounded buffering: records produced by the upstream during the migration window accumulate safely and are consumed by the target-tier instance in Phase 4. Simultaneously, each upstream records the Kafka offset

of the last record it delivered to o_i's original input topic prior to the PAUSE, the *drain offset* ω_{drain}. Phase 1 completes once o_i's hosting HEA confirms that its input topic consumer has processed all records up to and including offset ω_{drain}, establishing an empty input queue and a well-defined checkpoint boundary.

Phase 2—State Checkpoint. With o_i's input queue confirmed empty, the hosting HEA (or Flink connector) invokes o_i's state serialization interface, which captures the operator's complete runtime state into a structured snapshot. The snapshot comprises four components: (a) all active window buffer contents, represented as an ordered sequence of (key, window-start, value) tuples; (b) all accumulator states for aggregate operators; (c) the current event-time watermark and any pending timers; and (d) the Kafka consumer offset ω_{snap} of the last processed record across all of o_i's input partitions. These components are serialized using MessagePack [49], a compact, schema-aware binary serialization format, via a HybridStream-maintained schema registry that maps each operator type to a versioned serialization descriptor. The schema registry ensures that snapshots produced by the Python-based HEA are deserializable by the Java-based Flink connector (and vice versa) without a shared runtime, incurring a serialization overhead of approximately 3–8% compared to Flink's native Kryo-serialized savepoints (quantified in Section 7). The snapshot is written atomically to the shared object store (MinIO in MEC deployments, Amazon S3 in cloud deployments) under a deterministic key derived from o_i's operator ID and the current migration sequence number, ensuring idempotent re-upload on retry. The Kafka offset ω_{snap} establishes the exact replay boundary: the target-tier instance begins

consuming from offset $\omega_{\text{snap}} + 1$, preserving exactly-once semantics without record-level deduplication.

Phase 3—Target Initialization. The AODE directs the target tier θ_{dst} to instantiate o_i using the operator's registered implementation class, unchanged in the cloud case (R6), and restore it from the snapshot retrieved from the shared object store. Restoration proceeds in the reverse order of serialization: watermark and timer state first (establishing the event-time context), then accumulator and window buffer state (restoring aggregate progress), then the Kafka consumer offset (positioning the consumer group). The restored instance is initialized into a *ready-not-running* state: it has reconstructed all operator state and positioned its input channel at $\omega_{\text{snap}} + 1$, but it has not yet begun consuming records. A readiness acknowledgment is returned to the AODE once the restore is confirmed complete.

Phase 4—Flow Redirect and Resume. Upon receiving the readiness acknowledgment from θ_{dst}, the AODE performs two concurrent actions: it updates the global placement routing table to direct all future upstream output for o_i to θ_{dst}, and it signals the target instance to transition from *ready-not-running* to *running*. The target instance begins by draining the migration buffer, consuming all records buffered during Phases 1–3 in the order they were written, before switching to the live input channel at θ_{dst}. Once the migration buffer is fully consumed (confirmed when the target instance's consumer offset advances past the offset of the last record written to the buffer during Phase 1), the AODE signals the source-tier instance to terminate and removes it from the active instance registry. The upstream operators resume forwarding output directly to the target tier via the updated

routing table.

The total migration latency experienced by downstream consumers, the interval between Phase 1 upstream pause and Phase 4 buffer drain completion, is dominated by Phase 2 state serialization and object store upload. Across the operator types evaluated in Section 6, snapshot sizes range from approximately 8 MB (stateless filter and map operators, whose state consists only of watermark and offset metadata) to 420 MB (multi-stream temporal join operators with 5-minute windows at 150k events/second ingest rate). At the 210 MB/s upload bandwidth sustained by the MinIO deployment in the evaluation testbed (Section 6.1), these bounds correspond to migration pause durations of approximately 38ms and 2.0 seconds respectively. For latency-critical operators (λ_i = critical), the AODE scoring model assigns them to edge nodes conservatively and migrates them only under extreme resource pressure; the brief pause is bounded and deterministic, preferable to the unbounded latency degradation caused by edge resource exhaustion. Section 7 reports empirically measured migration pause durations for each scenario and operator type.

Fault tolerance. The PCTR protocol is designed to be idempotent with respect to failures at any phase. During Phase 1, if a PAUSE directive is lost or an upstream HEA fails mid-phase, the AODE retries the directive; no state has been modified on either tier, and the retry is safe. During Phase 2, if the HEA fails after writing the snapshot but before acknowledging the AODE, the snapshot remains valid in the object store and Phase 2 can be retried by a replacement HEA that resumes the source-tier instance from its most recent periodic checkpoint. During Phases 3–4, if the target instance fails

before completing the buffer drain, the AODE re-executes the protocol from Phase 3 using the same snapshot—the source-tier instance remains active and processing throughout Phases 3 and 4 as a fallback, and it is terminated only after Phase 4 completion is positively confirmed. This design ensures that at no point is there a window during which neither the source nor the target instance is actively processing o_i's stream segment.

5. Implementation

This section describes the implementation of HybridStream's three principal software components: the HybridStream Edge Agent (Section 5.1), the AODE service (Section 5.2), and the HybridStream Flink Connector (Section 5.3). Section 5.4 describes the cross-component communication substrate shared by all three. The complete reference implementation is made publicly available in the HybridStream repository to support reproducibility of the evaluation results in Section 7.

5.1. HybridStream Edge Agent

The HybridStream Edge Agent (HEA) is a Python 3.12 daemon process deployed as a single Docker container per edge node. It realizes the edge execution tier described in Section 4.1 through three integrated subsystems: an operator execution engine (Section 5.1.1), a state management layer (Section 5.1.2), and a gRPC management interface exposed to the AODE (Section 5.1.3). Section 5.1.4 characterizes the HEA's memory footprint relative to Apache Flink.

5.1.1. Operator Execution Engine

The HEA executes stream operators using a hybrid concurrency model designed to address CPython's Global Interpreter Lock (GIL) constraint, which prevents true thread-level parallelism for CPU-bound workloads within a single Python process, while avoiding the overhead of a full multi-process stream processing framework.

Operator classification. At registration time, each operator class is annotated as either I/O-bound or CPU-bound via a `@cpu_bound` decorator. I/O-bound operators, including stateless filters, map transformations, key-based routers, and lightweight rolling aggregations, execute as asyncio coroutines within the HEA's primary event loop, exploiting cooperative multitasking without incurring GIL contention. CPU-bound operators, including temporal join evaluations, machine learning inference tasks, and aggregations over large windows exceeding 60 seconds, are dispatched to a `concurrent.futures.ProcessPoolExecutor` with a pool size of max(1, N_cpu − 1), reserving one physical core for the event loop and gRPC server. Records are dispatched to worker processes via `asyncio.run_in_executor`, and completed results are resolved as futures within the event loop. This model achieves true CPU parallelism for compute-intensive operators while maintaining a unified asyncio execution context for I/O coordination. The inter-process serialization round-trip overhead for dispatched records, measured as 0.8–2.1 μs per record at the evaluated payload sizes, is reported in full in Section 7.3.

Kafka consumer integration. Each HEA maintains a dedicated `aiokafka` [50] asynchronous consumer group for its assigned Kafka input topics. The

consumer loop runs as an asyncio coroutine, polling in non-blocking batches of up to 500 records per iteration (configurable via `kafka_batch_size`). Consumed records are routed to the appropriate operator coroutine or worker dispatch queue based on a topic-to-operator binding table, which is updated atomically by the gRPC management interface upon receipt of an `ApplyPlacement` directive from the AODE.

Backpressure handling. When a CPU-bound operator's worker queue depth exceeds a configurable high-water mark (default: $2 \times$ pool size), the HEA suspends Kafka consumption for the affected topic partitions by pausing the `aiokafka` consumer assignment, leveraging Kafka's consumer lag mechanism as a natural backpressure buffer. Consumption resumes when the queue drains below the low-water mark. This mechanism bounds memory growth under transient CPU saturation without dropping records or violating Kafka's ordering guarantees.

5.1.2. State Management

Each operator hosted by the HEA is allocated a dedicated RocksDB column family within a shared RocksDB instance, accessed via the `rocksdict` Python bindings (version 0.8, RocksDB 8.10). Column family isolation ensures that write amplification from one operator does not degrade the read latency of co-hosted operators. The RocksDB instance is configured with 64 MB write buffers, a 256 MB block cache shared across all column families, and LZ4 block compression, providing a balance of write throughput and read performance suited to the access patterns of windowed aggregation state.

Periodic checkpointing. Every 30 seconds (configurable via `checkpoint_interval_s`),

the HEA initiates a lightweight background checkpoint for each hosted operator by invoking RocksDB's `CreateCheckpoint()` API. This API produces a point-in-time snapshot of the column family's SST (Sorted String Table) files via filesystem hard links, incurring negligible I/O overhead, measured at under 5ms per operator across all evaluated state sizes, and serving as recovery anchors for HEA restart scenarios.

Migration snapshot. When the AODE triggers Phase 2 of the PCTR protocol via the `TriggerSnapshot` RPC, the HEA serializes the operator's complete state into a MessagePack binary document. The serializer iterates over the operator's RocksDB column family entries, groups them by state type (window buffers, accumulators, watermarks, Kafka consumer offsets) according to the schema registry entry for the operator's class (Section 5.4), and writes a schema-versioned MessagePack document that embeds the schema version as a header field. The serialized document is uploaded asynchronously to the shared object store via `aioboto3` [51], an asyncio-compatible boto3 wrapper, under a deterministic object key of the form `{operator_id}/{migration_seq_num}/snapshot.msgpack`. The object key and compressed byte size are returned to the AODE in the `SnapshotResponse` message to enable Phase 3 target initialization.

5.1.3. gRPC Management Interface

The HEA exposes a gRPC server on a configurable TCP port (default: 50051) using the `grpcio` library (version 1.62) with a Python stub generated from the HybridStream proto3 service definition by `grpcio-tools`. The gRPC server runs on a dedicated background `ThreadPoolExecutor` with one thread, keeping it independent from the asyncio event loop and preventing

gRPC network I/O from blocking operator processing. The `HEAManagement` proto3 service exposes four RPC methods:

Listing 1: HEAManagement gRPC service definition (proto3).

```
service HEAManagement {
  rpc GetTelemetry     (TelemetryRequest)   returns (
     TelemetryResponse);
  rpc ApplyPlacement   (PlacementDirective) returns (
     PlacementAck);
  rpc TriggerSnapshot  (SnapshotRequest)    returns (
     SnapshotResponse);
  rpc TerminateOperator(TerminateRequest)   returns (
     TerminateAck);
}
```

`GetTelemetry` returns a `TelemetryResponse` containing the current CPU utilization, memory utilization, per-operator p95 processing latency, and ingest rate, each maintained as exponentially weighted moving averages (EWMA) by a background metric collection coroutine sampling at 5-second intervals. `ApplyPlacement` delivers an updated operator placement table from the AODE, atomically updating the HEA's routing state and initiating operator instantiation or graceful shutdown as needed. `TriggerSnapshot` initiates Phase 2 of the PCTR migration protocol, as described above. `TerminateOperator` is invoked by the AODE after Phase 4 migration confirmation to close the operator's asyncio coroutine or worker processes, flush and close its RocksDB column family, and release its Kafka consumer partition assignments.

5.1.4. Memory Footprint

Table 3 presents the measured baseline resident set size (RSS) of the HEA process compared to a minimal Apache Flink TaskManager deployment with one task slot, both running on idle nodes with no operators assigned. RSS values are medians of ten measurements taken after 60 seconds of idle warm-up.

Table 3: HEA vs. Apache Flink TaskManager baseline memory footprint (RSS, idle, no operators). Intel NUC 12 Pro, 60 s idle warm-up.

Component	RSS (MB)	Notes
CPython 3.12 interpreter	24.1	No application imports
+ grpcio 1.62 + protobuf 4.25	14.6	gRPC server, stub loaded
+ rocksdict 0.8 / RocksDB 8.10	11.3	One empty column family
+ aiokafka 0.10 + asyncio loop	5.2	Consumer registered
HEA total (baseline)	**55.2**	All subsystems initialized
Apache Flink TM (1 slot)	1,187	JVM 17, Flink 1.18.1
Memory reduction	**95.3%**	

As Table 3 shows, the HEA baseline occupies 55.2 MB of resident memory, a 95.3% reduction compared to the 1,187 MB consumed by the minimal Flink TaskManager. This reduction is attributable entirely to the elimination of JVM heap initialization, the Flink class loader (approximately 380 MB of framework classes), and the Flink network buffer pool (512 MB by default). On an edge node with 4 GB of available RAM, this footprint allows the co-hosting of up to 72 HEA baseline instances, versus approximately 3

Flink TaskManagers, before operator state consumption is factored in, substantially expanding the range of workloads executable at the edge tier.

5.2. AODE Service

The AODE service is a Python 3.12 asyncio application deployed as a Docker container on a low-latency virtual machine co-located with or proximate to the edge cluster. It implements the placement recalibration loop of Algorithm 1 (Section 4.2.3) through four cooperating asyncio components: the Telemetry Collector, the Placement Optimizer, the Migration Dispatcher, and the HA Coordinator.

5.2.1. Recalibration Loop

Telemetry Collector. At each recalibration cycle, the Telemetry Collector issues parallel gRPC `GetTelemetry` RPCs to all registered HEAs via `asyncio.gather()`, collecting all responses concurrently with a per-HEA timeout of 1 second. Collected metric values are stored in a circular buffer of depth 300 per metric (covering 25 minutes at the default 5-second interval). Ingest rate trend detection is performed using `scipy.stats.linregress` [46] over the trailing 12 buffer entries corresponding to the most recent 60 seconds. The OLS regression and p-value computation complete in approximately 0.3ms on the AODE host used in the evaluation (detailed in Section 6.1).

Placement Optimizer. The Placement Optimizer assembles the scoring matrix $\Phi \in \mathbb{R}\hat{}(|O| \times (|E|+1))$ using vectorized NumPy [52] array operations, evaluating all (operator, tier) pairs in a single pass. Infeasible assignments, those violating the capacity constraints of Section 3.2, are masked to $+\infty$

using a boolean feasibility matrix computed from current utilization and operator resource profiles. Topological ordering of the operator DAG, required by Algorithm 1, is computed using Kahn's algorithm [53] and cached between recalibration cycles; the cache is invalidated only when the DAG topology changes as a consequence of an active operator migration. For the workloads evaluated in Section 6 (up to 30 unique operator types, 4 HEA edge nodes), the complete scoring pass completes in 18–36ms on the AODE evaluation host, as measured in Section 7.3.

Migration Dispatcher. Operators flagged for migration by the Placement Optimizer are processed by the Migration Dispatcher as independent asyncio `Task` instances, one per migrating operator. Each task executes the four PCTR phases sequentially, awaiting AODE acknowledgment at each phase boundary. Tasks for operators that share a topological dependency are sequenced via asyncio `Event` objects, upstream migration tasks signal completion before dependent downstream tasks proceed to Phase 1, preserving the topological migration order required by Algorithm 1. After each PCTR phase transition, the task checkpoints its current state (phase, snapshot object key, drain offset, target readiness status) to etcd under a migration-specific key, enabling AODE restart recovery without re-initiating from Phase 1.

5.2.2. High Availability

The AODE HA mechanism relies on etcd's distributed lease primitive [45]. The primary AODE instance acquires an etcd lease with a time-to-live of 10 seconds, refreshed every 3 seconds by a background heartbeat coroutine. The standby instance monitors the lease expiry via an etcd watch stream. Upon

detecting lease expiry, the standby contests the lease via etcd's compare-and-swap operation; upon successful acquisition, it reads the current placement assignment and any in-progress migration states from etcd, reconstructs the in-memory recalibration state, and resumes the recalibration loop. Failover from primary failure to standby resumption completes within one recalibration interval ($\leq$5 seconds) in the worst case, bounded by the etcd lease TTL and the recalibration loop startup time.

5.3. HybridStream Flink Connector

The HybridStream Flink Connector is a Java 17 library loaded as a standard Apache Flink plugin via the Java `ServiceLoader` mechanism, requiring no modifications to application operator code and no changes to the Flink cluster configuration. It implements two responsibilities: receiving and processing AODE placement directives (Section 5.3.1), and translating between Flink's native savepoint format and HybridStream's MessagePack snapshot format (Section 5.3.2).

5.3.1. AODE Integration

The connector registers a gRPC server on the Flink JobManager using a Java stub generated from the same `hybridstream.proto` service definition (compiled by `protoc` [54], the Protocol Buffers compiler) as the Python HEA stubs. This design reuse means the AODE issues identical `GetTelemetry`, `ApplyPlacement`, `TriggerSnapshot`, and `TerminateOperator` RPCs regardless of whether the recipient is a Python HEA or the Java connector, eliminating tier-specific control plane logic in the AODE. `ApplyPlacement` directives from the AODE are translated into Flink `JobGraph` rescaling operations

by the connector's `PlacementDirectiveHandler`, which invokes the Flink `JobMasterGateway` REST API to adjust operator parallelism and task slot assignments. `TerminateOperator` triggers a Flink `cancel()` on the affected operator subtask.

5.3.2. Snapshot Translation

When the AODE issues a `TriggerSnapshot` RPC to the Flink connector, the connector invokes Flink's `CheckpointCoordinator` to produce a native savepoint, then translates the savepoint into the HybridStream MessagePack format in three steps: (1) it iterates over the savepoint's operator state handles via the Flink State Backend API; (2) it deserializes each state entry from Kryo-serialized bytes using the operator's registered `TypeSerializer`; and (3) it re-serializes the deserialized Java objects to MessagePack using the schema adapter registered for that operator class in the HybridStream schema registry (Section 5.4). The snapshot translation introduces a one-time overhead of 45–120ms per migration, dependent on operator state size, added to the Phase 2 upload time described in Section 4.3. This overhead applies only to cloud-to-edge migrations, where the Flink connector must translate a native Flink savepoint into the HybridStream MessagePack format; edge-to-cloud migrations bypass this step, as the HEA generates MessagePack snapshots directly. Measured translation times per direction are reported in Section 7.3.

5.4. Cross-Component Communication Protocol

All three HybridStream components communicate over a single shared proto3 service definition (`hybridstream.proto`), compiled to Python stubs

via `grpcio-tools` and to Java stubs via `protoc`. The proto3 schema constitutes the authoritative API contract for the HybridStream control plane; breaking changes require a service version increment, enforced by the schema registry version tag.

MessagePack schema registry. The schema registry is implemented as a set of versioned JSON documents stored under the well-known etcd key prefix `/hybridstream/schema/v1/<operator_class_name>`. Each document specifies the field names, field types, and serialization ordering of the MessagePack fields for a given operator class and schema version. Schema documents are written once at deployment time and thereafter treated as immutable; backward-incompatible operator state changes require a new schema version. Both the Python HEA (via the `msgpack` library [55]) and the Java Flink Connector (via the `msgpack-core` Java library [56]) resolve the schema for a given snapshot by reading the version tag embedded in the MessagePack document header and fetching the corresponding schema document from etcd. This lookup is performed once per operator class per process lifetime and cached in memory, imposing no per-record overhead during normal operation. The schema registry architecture ensures that a snapshot produced by any HEA or Flink instance can be consumed by any other, regardless of implementation language, enabling bidirectional migrations across the tier boundary without format asymmetry.

6. Experimental Evaluation

This section describes the experimental methodology used to evaluate HybridStream. Section 6.1 presents the evaluation platform, Section 6.2 defines

the three workloads derived from the motivating scenarios of Section 3.3, Section 6.3 describes the two baseline configurations against which Hybrid-Stream is compared, Section 6.4 defines the performance metrics collected, and Section 6.5 describes the experimental protocol, repetition strategy, and statistical analysis approach.

6.1. Evaluation Platform

All experiments were conducted on a physical testbed composed of three logical layers: an edge cluster, a cloud tier, and a simulated wide-area network (WAN) link connecting them.

Edge cluster. The edge tier comprised six Intel NUC 12 Pro nodes (NUC12WSHi5), each equipped with an Intel Core i5-1240P processor (12 cores: 4 P-cores at 4.4 GHz, 8 E-cores at 3.3 GHz), 16 GB LPDDR5-4800 RAM, and a 512 GB NVMe SSD. Nodes were connected via a dedicated 1 Gbps Ethernet switch with measured intra-cluster latency of 0.18 ± 0.03 ms (mean $\pm$ standard deviation over 10,000 ICMP probes). Each node ran Ubuntu 22.04 LTS (kernel 5.15), Docker 25.0.3, and Python 3.12.2. Four nodes served as active HEA nodes; two dedicated nodes hosted the MinIO distributed object store, providing sustained write throughput of 210 MB/s—the value referenced in Section 4.3. The AODE service was not hosted on any NUC node; its deployment is described in the AODE host paragraph below.

Cloud tier. The cloud tier was hosted on a single AWS c5.4xlarge instance (16 vCPU, 32 GB RAM, 10 Gbps ENA network), running Apache Flink 1.18.1 with four TaskManagers (2 task slots each, JVM heap 6 GB per TaskManager) and Apache Kafka 3.6.1 (3-broker cluster on the same in-

stance). The AODE service was co-located on the cloud instance for cloud-tier management; a separate t3.medium instance (2 vCPU, 4 GB RAM) hosted the AODE when evaluating edge-heavy placements, to validate the R5 overhead bound under deliberately constrained AODE resources.

WAN emulation. Wide-area network conditions between the edge cluster and cloud tier were emulated using the Linux `tc netem` (network emulator) kernel module applied to the egress interface of the edge switch uplink. Three network condition profiles were evaluated, as summarized in Table 4.

Table 4: Network condition profiles. RTT = one-way delay × 2. Jitter: Pareto distribution via `tc netem`.

Profile	Delay	RTT	Jitter	Loss	Represents
N1. Low	5 ms	10 ms	±0.5 ms	0%	MEC, 5G mmWave
N2. Medium	25 ms	50 ms	±3 ms	0.01%	Regional DC, LTE
N3. High	75 ms	150 ms	±12 ms	0.05%	Cross-region WAN

AODE host. During edge-heavy workloads (W1, W2), the AODE was hosted on the t3.medium instance (2 vCPU, 4 GB RAM) to evaluate overhead under constrained resources. During the financial workload (W3), which requires more frequent recalibration cycles due to ingest rate volatility, the AODE was co-located on the c5.4xlarge cloud instance to isolate placement decision latency from network effects. All AODE measurements distinguish between these two configurations in Section 7.

6.2. Workload Definitions

Three workloads were designed to correspond directly to the three motivating scenarios of Section 3.3. Each workload is specified as a Kafka-backed

operator DAG implemented using the HybridStream operator API and deployed identically across all three configurations (HybridStream, B1, B2) to ensure a controlled comparison.

W1. Industrial IoT Defect Detection. Workload W1 emulates Scenario 1: a high-speed manufacturing line with 200 vibration and thermal sensors each streaming at 10 kHz. Raw sensor records are ingested via a Kafka topic partitioned by sensor zone (40 sensors per partition, 5 partitions). The operator DAG comprises 12 operators: 5 per-zone raw signal normalizers (λ = standard, stateless), 5 parallel per-zone feature aggregation windows (30-second tumbling, λ = standard), a multi-stream join correlating inter-zone thermal patterns (λ = standard, state size $\sim$180 MB at peak), and a downstream binary classification operator emitting go/no-go decisions (λ = critical, SLO = 5 ms). Total sustained ingest rate: 2 million events per second. Three load phases are injected during each experiment run: normal load (100%), sustained peak (180% for 10 minutes), and spike (300% for 90 seconds). Each load transition constitutes an opportunity for AODE-triggered migration.

W2—Smart City Traffic Analytics. Workload W2 emulates Scenario 2: a traffic management system processing camera feeds from 500 intersections. The operator DAG comprises three operator types: a vehicle detection operator (λ = critical, SLO = 2 s, 500 parallel instances distributed across HEA nodes), a zone-level flow aggregation operator (λ = standard, 20 parallel instances), and a city-wide incident pattern detector (λ = batch, 1 instance, state size $\sim$380 MB). The AODE manages these three operator types as placement units, assigning all instances of each type collectively to a tier,

yielding a scoring matrix of 3 rows $\times$ 5 columns (4 HEA + cloud) at each recalibration cycle. Ingest rate: 50,000 detection events per second at baseline, rising to 95,000 during simulated rush-hour spikes. Memory pressure on edge nodes is the primary stress condition: zone aggregator instances are sized collectively to consume 85% of available edge RAM at peak load without migration, triggering the AODE's proactive memory-pressure migration behavior (Scenario 2, Section 3.3).

W3—Financial Market Stream Processing. Workload W3 emulates Scenario 3: tick-by-tick market data from 30 exchanges replayed from a recorded trace of the 2023 NYSE/NASDAQ consolidated tape, totaling 500,000 events per second at baseline. The operator DAG comprises 8 risk check operators (λ = critical, SLO = 1 ms each), 4 anomaly detection operators (λ = critical, SLO = 5 ms), 12 statistical aggregation operators (λ = standard), and 6 compliance logging operators (λ = batch). Three ingest rate spike events, sourced from recorded news-driven volatility periods in the trace, triple the ingest rate within a 10-second window, constituting the primary stress condition for this workload and testing the anticipatory trend-based migration behavior of the AODE.

Table 5 summarizes the key parameters of all three workloads.

6.3. Baseline Configurations

HybridStream is evaluated against two baseline configurations. Both baselines run identical operator DAGs on the same hardware using the same Kafka ingestion infrastructure; only the placement policy differs.

B1. Cloud-only Apache Flink. All operators execute as standard Flink tasks on the c5.4xlarge cloud instance. No edge processing occurs; all

Table 5: Workload summary. "Critical ops" = operators with λ = critical. State size at baseline ingest rate.

Workload	Ops	Critical	Base rate	Peak rate	State	Stress
W1. Industrial IoT	12	1 (5 ms)	2M ev/s	6M (300%)	∼210 MB	CPU
W2. Smart City	521	500 (2 s)	50k ev/s	95k (190%)	∼430 MB	Memory
W3. Financial	30	12 (1–5 ms)	500k ev/s	1.5M (300%)	∼165 MB	Ingest spikes

raw event data is forwarded from the edge Kafka cluster to the cloud tier for processing. This baseline represents the status quo for organizations using Flink today and characterizes the latency floor imposed by WAN round-trip times. B1 is run under all three network condition profiles (N1, N2, N3) to quantify the sensitivity of cloud-only latency to WAN conditions.

B2. Static hybrid. Operators are partitioned between edge and cloud tiers according to a fixed placement determined by a one-time offline assignment that minimizes projected end-to-end latency under average load conditions, using the same multi-factor scoring model as the HybridStream AODE evaluated at the baseline ingest rate with N2 (50ms RTT) network conditions. This placement is fixed for the entire experiment run and is not adjusted in response to runtime changes in resource availability, network conditions, or ingest rate. B2 represents the best achievable performance of a hybrid system without runtime adaptivity and directly isolates the contribution of the AODE's continuous recalibration to HybridStream's performance.

6.4. Performance Metrics

Six metrics were collected across all experiments:

- **M1—End-to-end p95 latency (ms):** The 95th-percentile time from record ingestion at the Kafka producer to output emission by the final operator in the DAG, measured per operator class (critical, standard, batch) using timestamp watermarks embedded in each event record at the Kafka producer. Reported separately for critical operators, the primary SLO-relevant metric, and aggregated across all operator classes.

- **M2. SLO compliance rate (%):** The proportion of 10-second measurement windows during which all critical operators sustain p95 latency at or below their defined SLO bounds. A window is marked non-compliant if any critical operator's p95 latency exceeds its SLO during that window. Reported as a percentage of total measurement windows per experiment run.

- **M3. Peak sustained throughput (events/s):** The maximum ingest rate at which the system processes all events without accumulating Kafka consumer lag exceeding 10 seconds of backlog at steady state. Measured by incrementally increasing the replay rate in 50,000 events/s steps and identifying the highest rate at which lag remains bounded for 5 continuous minutes.

- **M4. Migration pause duration (ms):** The elapsed time between Phase 1 upstream pause initiation and Phase 4 migration buffer drain completion (Section 4.3), measured per migration event. Reported as median and interquartile range (IQR) across all migration events in a run. Not applicable to B1 and B2, which perform no migrations.

- **M5. AODE recalibration overhead:** CPU utilization (%) and

memory resident set size (RSS, MB) of the AODE process on its host VM, sampled at 1-second intervals throughout each experiment run. Reported as mean ± standard deviation.

- **M6. Edge resource utilization (%):** Mean CPU and memory utilization across all active HEA nodes, sampled at 5-second intervals. Reported per workload phase (normal, spike, recovery) to characterize the AODE's effectiveness at maintaining edge resource headroom.

6.5. Experimental Protocol

Experiment structure. Each experiment consists of a single (workload, configuration, network profile) triple. Workloads W1 and W3 are each evaluated under all three network profiles (N1, N2, N3); W2 is evaluated under N2 only, as its primary stress condition, memory pressure, is network-independent. This yields $3 \times 2 \times 3 + 3 \times 1 \times 1 = 21$ unique (workload, configuration, network profile) triples across the three evaluation systems (HybridStream, B1, B2), for 63 total run configurations. Each configuration is repeated ten times, yielding 630 total experiment executions.

Run protocol. Each run follows a fixed timeline: a 10-minute warm-up period (during which the system reaches steady-state operator placement and Kafka consumer group stabilization, and measurements are discarded), followed by a 60-minute measurement window. Load injection (normal $\rightarrow$ spike $\rightarrow$ recovery) is scripted using a Kafka producer replay controller with deterministic timing, ensuring identical load profiles across all configurations and runs.

Repetitions and statistical analysis. Each of the 63 run configurations is executed ten times with independent random seeds for Kafka partition

assignment and worker process initialization. Results are reported as median values with 95% bootstrap confidence intervals (CI), computed using 10,000 bootstrap resamples over the ten run medians [57]. Pairwise comparisons between HybridStream and each baseline (B1, B2) are evaluated using the Wilcoxon signed-rank test [58]. a non-parametric test appropriate for latency distributions, which are right-skewed and non-Gaussian. with a significance threshold of $\alpha = 0.05$ after Bonferroni correction for multiple comparisons. Effect sizes are reported as rank-biserial correlation coefficients r, with $|r| > 0.5$ considered large.

Research questions. The experimental evaluation is structured around three research questions, each mapping to a class of claims made in Section 1 and the design requirements of Section 3.4: - **RQ1:** Does HybridStream sustain SLO compliance for latency-critical operators under the three stress conditions and all evaluated network profiles? - **RQ2:** Does continuous adaptive placement improve SLO compliance and latency over a statically configured hybrid baseline (B2)? - **RQ3:** What are the resource and latency overheads of HybridStream's adaptive machinery (AODE recalibration, state migration) relative to operator execution?

Reproducibility. All workload traces, operator implementations, Kafka topic configurations, Docker container images, and experiment orchestration scripts are included in the HybridStream reference repository. The W3 financial trace is derived from NYSE/NASDAQ consolidated tape data for the trading day 2023-10-10, obtained under a standard market data license, processed with a deterministic replay filter to reproduce ingest rate spikes at fixed timestamps. Complete reproduction instructions are provided in the

repository README.

7. Results & Discussion

This section presents and interprets the experimental results. Section 7.1 addresses RQ1: whether HybridStream sustains SLO compliance for latency-critical operators under variable network conditions and load patterns. Section 7.2 addresses RQ2: whether continuous adaptive recalibration improves over a statically partitioned hybrid baseline. Section 7.3 addresses RQ3: the overhead cost of HybridStream's adaptive machinery. Section 7.4 discusses the cross-cutting findings, identifies limitations, and examines threats to validity. Throughout, results are reported as median values with 95% bootstrap confidence intervals; statistical significance is determined by the Wilcoxon signed-rank test at $\alpha = 0.05$ after Bonferroni correction, with rank-biserial correlation r as the effect size measure.

7.1. RQ1: SLO Compliance and Latency Under Variable Conditions

Finding 1: Cloud-only deployment (B1) fails to satisfy any latency-critical SLO across all workloads and all network profiles.

Table 6 presents the p95 end-to-end processing latency for critical operators across all three workloads and network profiles under peak load conditions. Under B1, the WAN round-trip time is the binding latency factor, and none of the evaluated SLOs, 5 ms (W1), 2 s (W2 under cloud overload), or 1 ms (W3 risk checks), are satisfied. As Table 6 illustrates, even under the most favorable network profile (N1, 10 ms RTT), the cloud-only p95 latency for W1's binary classifier reaches 9.4 ms, 88% above its 5 ms SLO, and for W3's risk check operators reaches 6.8 ms, 580% above their 1 ms

SLO. Under N3 (150 ms RTT), cloud-only latency for both workloads exceeds 80 ms, rendering SLO compliance structurally impossible regardless of cloud-tier processing speed. The SLO compliance rate (M2) for B1 is 0.0% for W1 and W3 across all network profiles, and 84.3% for W2 under N2, partially compliant only because the W2 SLO of 2 seconds is large enough to accommodate cloud round-trip time at baseline load, but violated during peak-load overload on the cloud TaskManagers.

These results confirm that cloud-only deployment is categorically unsuitable for latency-critical stream processing under any of the evaluated SLO regimes, even under low-latency network conditions. The WAN round-trip time alone, independent of cloud processing time, exceeds the 1 ms and 5 ms SLOs by 5× to 90×, establishing an absolute lower bound on cloud-only latency that no amount of cloud compute scaling can overcome without reducing the physical distance between client and processor.

Finding 2: HybridStream sustains SLO compliance at 99.2–99.8% of measurement windows, maintaining compliance during load spikes that cause B1 and B2 to violate their respective SLOs.

Table 7 presents the SLO compliance rates (M2) for all workloads and configurations across load phases. As Table 7 shows, HybridStream achieves 99.2–99.8% compliance across all workloads and network profiles. B2 achieves 84.3–97.2% compliance depending on workload; compliant under normal load conditions but failing during the stress conditions specific to each workload's design. The difference in compliance rate between HybridStream and B2 represents the quantified value of runtime adaptivity.

All HybridStream vs B2 compliance comparisons are statistically signif-

Table 6: M1: Critical operator p95 latency (ms) at peak load. Medians over 10 runs. Bold = SLO compliant.

W1. Classifier (SLO = 5 ms)

Configuration	N1 (10 ms)	N2 (50 ms)	N3 (150 ms)
B1 (cloud-only)	9.4 ×	34.1 ×	91.7 ×
B2 (static hybrid)	1.8 ✓	1.9 ✓	2.1 ✓
HybridStream	**1.6** ✓	**1.7** ✓	**1.9** ✓

B2 rises to 8.4–9.7 ms during 300% spikes; HybridStream sustains 1.6–2.1 ms.

W3. Risk check (SLO = 1 ms)

Configuration	N1	N2	N3
B1 (cloud-only)	6.8 ×	27.3 ×	81.4 ×
B2 (static hybrid)	0.4 ✓	0.4 ✓	0.4 ✓
HybridStream	**0.3** ✓	**0.3** ✓	**0.4** ✓

W3. Anomaly detection (SLO = 5 ms)

Configuration	N1	N2	N3
B1 (cloud-only)	8.9 ×	30.1 ×	85.2 ×
B2 (static hybrid)	1.2 ✓	1.3 ✓	1.4 ✓
HybridStream	**1.1** ✓	**1.2** ✓	**1.3** ✓

Table 7: M2: SLO compliance rate (%). Medians over 10 runs with 95% bootstrap CI.
Wilcoxon: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

Workload	Configuration	N1	N2	N3
W1	B1 (cloud-only)	0.0%	0.0%	0.0%
W1	B2 (static hybrid)	97.2 (96.4–97.9)	97.1 (96.3–97.8)	96.8 (96.0–97.6
W1	**HybridStream**	**99.8 (99.6–99.9)*****	**99.7 (99.5–99.9)*****	**99.4 (99.1–99.6)**
W2	B1 (cloud-only)	,	84.3 (83.1–85.4)	,
W2	B2 (static hybrid)	,	92.4 (91.6–93.1)	,
W2	**HybridStream**	—	**99.2 (98.9–99.5)*****	—
W3	B1 (cloud-only)	0.0%	0.0%	0.0%
W3	B2 (static hybrid)	95.1 (94.2–96.0)	95.3 (94.4–96.2)	95.4 (94.5–96.3
W3	**HybridStream**	**99.7 (99.5–99.8)*****	**99.6 (99.4–99.8)*****	**99.5 (99.2–99.7)**

icant at $p < 0.001$ after Bonferroni correction, with large effect sizes (r = 0.88–0.94). The 2.4–6.8 percentage-point improvement over B2 translates to a 3–12× reduction in the number of non-compliant windows per hour (from 103–217 windows per hour for B2 to 12–48 for HybridStream across workloads), representing a materially different operational profile for time-sensitive applications. The W1 and W3 improvements are driven by the AODE's CPU-load and ingest-trend responses, respectively; the W2 improvement is driven by the proactive memory-pressure response, consistent with the three failure modes formalized in Section 3.3.

Finding 3: HybridStream's SLO compliance is robust to network condition degradation, declining by at most 0.4 percentage points from N1 to N3.

The marginal sensitivity of HybridStream's SLO compliance to network condition (0.4 pp across the 140 ms RTT range from N1 to N3) demonstrates that the AODE's scoring model effectively accounts for varying inter-tier transfer costs when making placement decisions: under high-RTT conditions, latency-critical operators are placed more aggressively on edge nodes, compensating for the increased network penalty on cross-tier data paths.

7.2. RQ2: Adaptive vs. Static Hybrid Placement

Finding 4: Continuous adaptive placement improves p95 latency for critical operators during stress periods by 76–86% compared to static hybrid (B2), while maintaining equivalent latency during normal-load periods.

The performance differential between HybridStream and B2 is negligible during normal load conditions (median latency difference < 0.2 ms across

all workloads); confirming that the AODE's steady-state placement is equivalent to a well-configured static partition. The divergence emerges during stress periods: under W1's 300% load spike, B2's classifier p95 latency rises to 8.4–9.7 ms while HybridStream's remains at 1.6–2.1 ms (76–78% lower). Under W3's ingest rate spike, B2's risk check operators rise to 2.1 ms while HybridStream's remain at 0.3–0.5 ms (76–86% lower). These improvements result directly from the AODE's proactive detection and response to emerging stress conditions through the trend-adjusted scoring model described in Section 4.2.2.

Finding 5: HybridStream sustains peak throughput within 3.1–5.7% of B2, demonstrating that adaptive placement does not materially reduce processing capacity.

Table 8 presents the peak sustained throughput (M3) for all workloads. As Table 8 shows, HybridStream's throughput is 3.1% below B2 for W1, 3.7% below for W2, and 5.7% below for W3—all within the 6% envelope stated in Section 1. The throughput reduction relative to B2 is attributable to three sources: (a) migration pause overhead during AODE-triggered migrations (∼0.4–2.0 s per migration, reported in Section 7.3), (b) MessagePack serialization overhead for cross-tier record transfer (3–8% per-record cost, negligible on aggregate throughput), and (c) the Kafka bridge topic round-trip for inter-tier operator communication (∼2 ms at N2). The W3 workload incurs the largest throughput reduction because it triggers the most migrations, three ingest rate spikes each triggering migration of 12–18 operators, accumulating more total migration pause time per run.

The B1 cloud-only baseline consistently achieves lower throughput than

Table 8: M3: Peak sustained throughput (events/s). Medians over 10 runs.

Workload	B1	B2	HybridStream	HS vs B2
W1	4,100,000	5,800,000	5,620,000	−3.1%
W2	71,000	82,000	79,000	−3.7%
W3	1,250,000	1,410,000	1,330,000	−5.7%

both hybrid configurations, as the cloud TaskManagers (8 slots across 4 TaskManagers on a single c5.4xlarge) are rate-limited by CPU contention well before the hybrid configurations' combined edge + cloud compute budget. This demonstrates that the throughput advantage of hybrid deployment is additive—edge and cloud compute complement rather than compete.

7.3. RQ3: Overhead of the Adaptive Machinery

Finding 6: AODE recalibration overhead is negligible on the evaluation hardware—below 2% CPU and 145 MB RSS on the constrained t3.medium host.

Table 9 presents the AODE recalibration overhead (M5) for each workload configuration. The recalibration CPU overhead of 0.8–1.8% and RSS of 134–142 MB on the t3.medium (2 vCPU, 4 GB RAM) host validate requirement R5: the AODE's resource consumption is negligible relative to operator execution on any practical edge or cloud host. The W3 workload exhibits the highest recalibration time (36.2 $\pm$ 4.8 ms median) due to its larger scoring matrix (30 operator types $\times$ 5 tiers = 150 entries), which is consistent with the $O(|O| \cdot (|E|+1))$ complexity analysis of Section 4.2.3 and remains well within the 5-second recalibration interval.

Table 9: M5: AODE recalibration overhead (mean ± SD, 10 runs).

Workload	Host	CPU (%)	RSS (MB)	Recalib. (ms)
W1	t3.medium	1.8 ± 0.3	142 ± 8	18.4 ± 3.1
W2	t3.medium	1.4 ± 0.2	134 ± 6	12.1 ± 2.0
W3	c5.4xlarge	0.8 ± 0.1	138 ± 7	36.2 ± 4.8

Finding 7: Migration pause duration scales linearly with snapshot size, with a median of 62 ms for small operators and 1,410 ms for large stateful operators.

Table 10 presents the migration pause durations (M4) categorized by operator state size. As Table 10 shows, pause duration ranges from a median of 62 ms for small operators (snapshot 8–15 MB) to 1,410 ms for large join operators (snapshot 200–420 MB). These values are consistent with the theoretical estimates derived in Section 4.3 (38 ms to 2.0 s at 210 MB/s upload bandwidth, based on Phase 2 upload time alone), with the offset between theory and measurement attributable to Phase 1 drain latency (∼12 ms median), Phase 3 state restore time (∼8 ms for small, ∼80 ms for large), Phase 4 migration buffer drain (∼6 ms for small, ∼60 ms for large operators), and, for cloud-to-edge migrations only, the Flink savepoint translation overhead (∼45–120 ms; Section 5.3.2). Edge-to-cloud migrations, which represent the majority of events in the evaluated workloads, do not incur this translation step.

The inter-process serialization round-trip overhead for CPU-bound operator record dispatch (M5 component) was measured at a median of 0.9 μs per record at the W3 evaluation payload sizes (average 128-byte tick records),

Table 10: M4: Migration pause duration (ms) by operator state size.

Category	State size	Median	IQR	Min–Max
Small	8–15 MB	62	8	48–91
Medium	50–200 MB	510	62	411–698
Large	200–420 MB	1,410	195	1,104–1,844

within the 0.8–2.1 μs range stated in Section 5.1.1. This overhead is negligible in absolute terms relative to the SLOs evaluated (1–2,000 ms).

No SLO violation was observed to result from a migration pause event in HybridStream across all 630 experiment executions. This is because the AODE's scoring model applies the hysteresis threshold and SLO urgency factor (Φ_{slo}, Section 4.2.2) to defer migrations for λ = critical operators until edge resource exhaustion makes continued edge placement infeasible; at which point the bounded migration pause is preferable to unbounded latency degradation from resource starvation.

7.4. Discussion

7.4.1. Scenario-Specific Analysis

W1 (CPU saturation. Scenario 1). The AODE's reactive migration response to CPU pressure (moving the zone feature aggregation operators to the cloud when edge utilization crosses the recalibration threshold) reduced classifier p95 latency during the 300% spike from 8.4–9.7 ms (B2) to 1.6–2.1 ms. The mean time from edge CPU threshold crossing to completed upstream operator migration was 13.2 ± 2.4 seconds across all W1 runs. approximately 2.6 recalibration cycles. The 10-minute sustained peak phase

exposed B2's fundamental limitation: static placement has no mechanism to trade non-critical edge operators for cloud resources when contention arises, leading to progressive classifier latency degradation as the aggregation window operators consume increasing fractions of available CPU.

W2 (memory pressure. Scenario 2). The proactive memory-pressure migration in W2 was triggered a median of 4.2 minutes before the edge memory high-water mark was reached, based on the AODE's Φ_{res} factor detecting zone aggregator memory growth trending toward the feasibility ceiling. This anticipatory response prevented SLO violation in 99.2% of measurement windows. compared to 92.4% for B2, which has no mechanism to respond to memory pressure before it manifests as processing backpressure. The 7.1-point compliance improvement for W2 (92.4% $\rightarrow$ 99.2%) is the largest single cross-configuration improvement observed across all workloads, reflecting the severity and predictability of memory-pressure degradation as a failure mode.

W3 (ingest rate spikes. Scenario 3). The AODE's trend-adjusted scoring detected the three recorded ingest rate spikes an average of 22.6 $\pm$ 3.1 seconds before the spike peak, activating forward-projected scoring with H = 30 second lookahead at a p-value of 0.031–0.048 across the 10 runs. Statistical aggregation operators were preemptively migrated to the cloud a median of 18.4 seconds before the spike would have saturated edge capacity. 3.6 seconds ahead of the spike onset. During runs where the trend was not detected sufficiently early (2 of 30 spike events across 10 runs), the AODE fell back to reactive migration, resulting in a 0.6-second window of SLO violation for risk check operators. This accounts for the residual 0.3% non-compliant windows reported for W3 in Table 7. The reactive fallback

behavior demonstrates that the AODE degrades gracefully when trend-based prediction fails, rather than catastrophically.

7.4.2. Limitations

Single cloud instance. All evaluations used a single AWS c5.4xlarge instance as the cloud tier. Real deployments may use multi-region Flink clusters, introducing additional complexity in operator placement across geographically distributed cloud tiers. The AODE architecture supports multiple cloud regions by treating each as a distinct tier in the scoring matrix, but this configuration was not evaluated. Multi-region cloud deployment is left as future work.

Python runtime performance ceiling. The HEA's Python runtime introduces a performance ceiling for CPU-bound operators that a compiled-language implementation (e.g., Rust or Go) would not. For W1 at 300% load (6 million events/second), four HEA nodes running CPU-bound classification operators at near-full utilization on 4 P-cores each approached their processing limit at peak. A Go or Rust HEA would handle higher ingest rates before triggering AODE migration, but with a higher implementation complexity. The Python choice prioritizes deployment simplicity and ecosystem access over raw throughput, a trade-off that is appropriate for the operator profiles evaluated but may not generalize to extreme-throughput deployments exceeding 10 million events/second per edge node.

Hysteresis threshold tuning. The $\Delta_h = 0.15$ hysteresis threshold was calibrated on 48-hour telemetry traces from the three evaluation workloads, as described in Section 4.2.3. Deployments with substantially different workload profiles, such as workloads with very high intra-window variance

or irregular operator dependency graphs, may require re-tuning this threshold. An automated threshold calibration mechanism that adapts Δ_h based on observed oscillation frequency is a direction for future work.

Edge node availability assumption. The evaluation assumes edge nodes operate reliably throughout each run. As noted in Section 3.1, handling edge node failures is treated as a system-level concern—the PCTR protocol designates unreachable HEAs for emergency cloud migration (Section 4.2.1), but the interaction between failure recovery and active PCTR migrations was not evaluated. Testing HybridStream's behavior under concurrent node failures and in-progress migrations is a direction for future work.

Operator state recovery time. Phase 3 state restoration for large operators ($\geq$200 MB snapshots) adds 80–260 ms to the migration pause, in addition to Phase 2 upload time. For deployments where even 1–2 second pauses are unacceptable (e.g., sub-millisecond SLO operators with large state), this implies that the AODE must prioritize keeping such operators on stable, well-resourced tiers. The current scoring model's conservative placement of critical operators (via the Φ_{slo} factor) achieves this in practice, but explicit migration eligibility constraints for λ = critical operators with large state would strengthen this guarantee.

7.4.3. Baseline Selection

The evaluation compares HybridStream against a cloud-only deployment (B1) and a statically partitioned hybrid (B2). A natural question is why no reinforcement learning (RL)-based offloading baseline is included, given the discussion in Section 2.4. We note that existing RL-based approaches [38, 39] target discrete task offloading in request-response workloads rather than

continuous stateful stream processing. Adapting an RL agent to the stream operator placement domain would require designing reward functions, state representations, and action spaces that do not yet exist in published work, constituting a separate research contribution. Similarly, NebulaStream [32] does not provide a publicly available benchmark harness for cross-system comparison. We therefore follow the methodology of Cardellini et al. [33], who compare against static and threshold-based baselines when no directly comparable adaptive system exists.

7.4.4. Threats to Validity

Internal validity. The primary threat to internal validity is the use of tc netem to emulate WAN conditions rather than a geographically distributed testbed. While tc netem accurately models the latency, jitter, and packet loss profiles specified in Table 4, it does not reproduce asymmetric routing, BGP rerouting events, or long-tail latency distributions that characterize real WAN links. The evaluation results may therefore represent optimistic estimates of B1 (cloud-only) compliance under real WAN conditions; real cloud-only deployments would likely exhibit higher non-compliance rates than those reported here, strengthening rather than weakening the case for hybrid edge-cloud deployment.

External validity. The three workloads evaluated were designed to be representative of broad application classes (industrial IoT, smart city, financial processing), but are implemented as synthetic operator graphs rather than production application code. The generalizability of the results to specific production deployments depends on the similarity of their operator DAG structure, state profile, and latency requirements to the evaluated workloads.

The HybridStream operator API is designed to accept unmodified application operator code (R6), but production operators may exhibit performance characteristics, such as irregular state size growth or bursty CPU profiles, not captured by the synthetic workloads.

Construct validity. The SLO compliance rate metric (M2) measures the fraction of 10-second windows in which all critical operators meet their SLOs, which may mask brief sub-window violations. A finer-grained metric, such as the fraction of individual records that experience end-to-end latency exceeding the SLO, would provide a more conservative assessment of SLO compliance, and is a direction for future measurement refinement.

8. Conclusion & Future Work

8.1. Summary of Contributions

This paper presented HybridStream, a hybrid edge-cloud stream processing framework that achieves low-latency real-time analytics through continuous, workload-adaptive operator placement. Four concrete contributions were made and empirically validated.

Adaptive placement model and scoring function. The Adaptive Offloading Decision Engine (AODE) places stream processing operators across edge and cloud tiers using the multi-factor scoring function (Eq. 1, Section 4.2.2) that integrates EWMA-smoothed latency, resource utilization, SLO urgency, and inter-tier transfer cost. Placement decisions are issued within 18–36 ms (measured on the evaluation platform) and are bounded by 200 ms worst-case, satisfying the lightweight decision overhead requirement R5. The trend-adjusted variant detected rising ingest rates an average of 22.6

$\pm$ 3.1 seconds before their peak, enabling preemptive migration before edge resources were exhausted.

Pause-checkpoint-transfer-resume (PCTR) migration protocol. The PCTR protocol transfers stateful operator execution between tiers with a bounded, predictable pause ranging from 62 ms for small-state operators to 1,410 ms for large join operators, consistent with the theoretical derivation in Section 4.3. Across all 630 experiment executions. spanning 21 unique (workload $\times$ configuration $\times$ network) triples, three systems, and ten repetitions each. zero SLO violations were attributable to a migration pause event, validating the protocol's safe integration into a live SLO-governed pipeline.

Lightweight edge runtime. The HybridStream Edge Agent (HEA) provides a Python 3.12 operator execution environment with a baseline memory footprint of 55.2 MB resident set size—a 95.3% reduction relative to an Apache Flink TaskManager (1,187 MB). This reduction enables co-hosting of up to 72 idle HEA instances on a 4 GB edge node, substantially expanding the range of operator configurations that can be maintained at the edge tier without infrastructure-scale hardware.

Experimental validation across three workload classes. Across workloads representing industrial IoT, smart city, and financial processing applications, evaluated under three network profiles (10, 50, and 150 ms RTT) and two load stress modes, HybridStream sustained SLO compliance at 99.2–99.8% of measurement windows. This represents a 2.4–6.8 percentage-point improvement over the static hybrid baseline (B2) at $p < 0.001$ (Wilcoxon, Bonferroni-corrected) with large effect sizes (r = 0.88–0.94), and a categorical improvement over cloud-only deployment, which achieved 0.0% compliance

for sub-10 ms SLOs across all network profiles. Peak throughput was sustained within 5.7% of B2. AODE recalibration overhead remained below 2% CPU and 145 MB RSS on a 2-vCPU host.

Reference implementation. A fully functional prototype of HybridStream. including the HEA, AODE service, Flink Connector, all workload operator implementations, experiment orchestration scripts, and the W3 trace replay filter. is publicly available at https://github.com/umur/hybrid-stream to enable full reproducibility of all reported results and to facilitate adoption and extension by the research community.

Taken together, these results establish that adaptive hybrid edge-cloud stream processing can simultaneously satisfy strict latency SLOs (1–5 ms for critical operators), maintain competitive throughput (within 6% of an optimally configured static baseline), and operate with negligible control-plane overhead; three properties that prior systems required separate dedicated deployments to achieve.

8.2. Future Work

Four directions follow directly from the limitations identified in Section 7.4.2.

Multi-region cloud tier. The AODE scoring model supports multiple cloud regions through additional columns in the placement matrix, but all evaluations used a single cloud region. Extending to geographically distributed Flink clusters would quantify the AODE's ability to exploit regional pricing and latency differences, and would expose new challenges in state consistency and migration routing across regions.

Compiled-language edge runtime. The Python 3.12 HEA approaches

its single-node throughput ceiling at approximately 10 million events per second for CPU-bound operators. A Rust or Go HEA would raise this ceiling substantially while preserving HybridStream's operator portability contract. The key design challenge is maintaining the inter-process isolation model that currently insulates the HEA control loop from operator-level faults under a compiled-language implementation.

Automated hysteresis calibration. The $\Delta_h = 0.15$ hysteresis threshold was calibrated offline. For deployments with high intra-window variance or irregular dependency structures, an online calibration mechanism that adapts Δ_h based on observed oscillation frequency and false-migration rate would reduce operator expertise requirements and improve robustness to distributional shift in production traffic.

Per-record SLO compliance tracking. The M2 metric measures SLO compliance at 10-second window granularity, which may mask brief sub-window violations. Instrumenting the HEA and Flink Connector to track per-record end-to-end latency would provide a more conservative and operationally relevant compliance signal, enabling tighter feedback between the AODE's scoring model and application-level outcomes.

Data Availability

The complete HybridStream reference implementation, including all source code, workload operator implementations, experiment orchestration scripts, Docker deployment configurations, and the W3 trace replay filter, is publicly available at `https://github.com/umur/hybrid-stream`.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author thanks the anonymous reviewers for their constructive feedback.

CRediT authorship contribution statement

Umur Inan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Umur Inan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] Ericsson, Ericsson mobility report, Tech. rep., Ericsson AB, Stockholm, Sweden (Jun. 2023).
URL https://www.ericsson.com/en/reports-and-papers/mobility-report/reports/june-2023

[2] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, K. Tzoumas, Apache Flink: Stream and batch processing in a single engine, IEEE Data Engineering Bulletin 38 (4) (2015) 28–38.

[3] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, I. Stoica, Discretized streams: Fault-tolerant streaming computation at scale, in: Proc. 24th ACM Symposium on Operating Systems Principles (SOSP), 2013, pp. 423–438. doi:10.1145/2517349.2522737.

[4] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. NetDB Workshop, 2011.

[5] J. George, Optimizing hybrid and multi-cloud architectures for real-time data streaming and analytics: Strategies for scalability and integration, World Journal of Advanced Engineering Technology and Sciences (2022).

[6] A. K. Immadisetty, Edge analytics vs. cloud analytics: Tradeoffs in real-time data processing, Journal of Recent Trends in Computer Science and Technology (2025).

[7] M. Satyanarayanan, The emergence of edge computing, Computer 50 (1) (2017) 30–39. doi:10.1109/MC.2017.9.

[8] ETSI, Multi-access edge computing (MEC): Framework and reference architecture, Tech. Rep. ETSI GS MEC 003, v3.1.1, European Telecommunications Standards Institute (2022).

[9] L. Liu, H. Li, M. Gruteser, Edge assisted real-time object detection for mobile augmented reality, Proc. 25th Annual International Conference on Mobile Computing and Networking (MobiCom) (2019). doi:10.1145/3300061.3300116.

[10] P. Mach, Z. Becvar, Mobile edge computing: A survey on architecture and computation offloading, IEEE Communications Surveys & Tutorials 19 (3) (2017) 1628–1656. doi:10.1109/COMST.2017.2682318.

[11] X. Wang, A. Khan, J. Wang, A. Gangopadhyay, An edge–cloud integrated framework for flexible and dynamic stream analytics, Future Generation Computer Systems 134 (2022) 1–14. doi:10.1016/j.future.2022.04.001.

[12] F. Nazir, Enhancing big data processing with a hybrid cloud-edge framework: Addressing latency, privacy, and scalability challenges in real-time analytics, International Journal of Applied Sciences and Society (2022).

[13] A. K. Pallikonda, A. K. Katragadda, J. Rao, Optimizing real-time IoT processing with hybrid edge-cloud architecture for enhanced latency and energy efficiency, Journal of Theoretical and Applied Information Technology 103 (2025).

[14] R. Alsurdeh, R. N. Calheiros, K. M. Matawie, Hybrid workflow scheduling on edge cloud computing systems, IEEE Access 9 (2021).

[15] S. Wang, Y. Zhao, J. Xu, J. Yuan, C.-H. Hsu, Edge server placement in mobile edge computing, Journal of Parallel and Distributed Computing 127 (2019) 160–168. doi:10.1016/j.jpdc.2018.06.008.

[16] A. Yousefpour, et al., All one needs to know about fog computing and related edge computing paradigms: A complete survey, Journal of Systems Architecture 98 (2019) 289–330. doi:10.1016/j.sysarc.2019.02.009.

[17] A. Toshniwal, S. Taneja, A. Shukla, K. Ramasamy, J. M. Patel, S. Kulkarni, J. Jackson, K. Gade, M. Fu, J. Donham, N. Bhagat, S. Mittal, D. Ryaboy, Storm@Twitter, in: Proc. ACM SIGMOD International Conference on Management of Data, 2014, pp. 147–156. doi:10.1145/2588555.2595641.

[18] J. Xu, Z. Chen, J. Tang, S. Su, T-Storm: Traffic-aware online processing in Storm, in: Proc. 34th IEEE International Conference on Distributed Computing Systems (ICDCS), 2014, pp. 535–544. doi:10.1109/ICDCS.2014.61.

[19] B. Peng, M. Hosseini, Z. Hong, R. Farivar, R. Campbell, R-Storm: Resource-aware scheduling in Storm, in: Proc. 16th

ACM/IFIP/USENIX International Middleware Conference, 2015, pp. 149–161. doi:10.1145/2814576.2814808.

[20] S. Kulkarni, N. Bhagat, M. Fu, V. Kedigehalli, C. Kellogg, S. Mittal, J. M. Patel, K. Ramasamy, S. Taneja, Twitter Heron: Stream processing at scale, in: Proc. ACM SIGMOD International Conference on Management of Data, 2015, pp. 239–250. doi:10.1145/2723372.2742788.

[21] B. Peng, M. Hosseini, Z. Hong, R. Farivar, R. Campbell, R-Storm: Resource-aware scheduling in Storm, in: Proc. 16th Annual Middleware Conference, 2015, pp. 149–161. doi:10.1145/2814576.2814808.

[22] M. Fragkoulis, P. Carbone, V. Kalavri, A. Katsifodimos, A survey on the evolution of stream processing systems, The VLDB Journal 32 (2023) 507–541. doi:10.1007/s00778-022-00749-x.

[23] M. Hirzel, R. Soulé, S. Schneider, B. Gedik, R. Grimm, A catalog of stream processing optimizations, ACM Computing Surveys 46 (4) (2014) 1–34. doi:10.1145/2528412.

[24] P. Pietzuch, J. Ledlie, J. Shneidman, M. Roussopoulos, M. Welsh, M. Seltzer, Network-aware operator placement for stream-processing systems, in: Proc. 22nd International Conference on Data Engineering (ICDE), 2006, p. 49. doi:10.1109/ICDE.2006.105.

[25] T. Heinze, V. Pappalardo, Z. Jerzak, C. Fetzer, Auto-scaling techniques for elastic data stream processing, in: Proc. IEEE International Conference on Data Engineering Workshops (ICDEW), 2014, pp. 296–302. doi:10.1109/ICDEW.2014.6818342.

[26] V. Kalavri, J. Liagouris, M. Hoffmann, D. Dimitrova, M. Forber, T. Roscoe, Three steps is all you need: Fast, accurate, automatic scaling decisions for distributed streaming dataflows, in: Proc. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018, pp. 783–798.

[27] A. Floratou, A. Agrawal, B. Graham, S. Rao, K. Ramasamy, Dhalion: Self-regulating stream processing in Heron, in: Proc. VLDB Endowment, Vol. 10, 2017, pp. 1825–1836. doi:10.14778/3137765.3137786.

[28] F. Bonomi, R. Milito, J. Zhu, S. Addepalli, Fog computing and its role in the Internet of Things, in: Proc. ACM MCC Workshop on Mobile Cloud Computing, 2012, pp. 13–16. doi:10.1145/2342509.2342513.

[29] W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu, Edge computing: Vision and challenges, IEEE Internet of Things Journal 3 (5) (2016) 637–646. doi:10.1109/JIOT.2016.2579198.

[30] H. Kuchuk, E. Malokhvii, Integration of IoT with cloud, fog, and edge computing: A review, Advanced Information Systems 8 (1) (2024).

[31] R. Ghosh, Y. Simmhan, Distributed scheduling of event analytics across edge and cloud, in: Proc. IEEE/ACM International Conference on Utility and Cloud Computing (UCC), 2017.

[32] S. Zeuch, E. Monterrey Viña, et al., NebulaStream: Complex analytics beyond the cloud, in: Proc. International Workshop on Very Large Internet of Things (VLIoT) at VLDB, 2020.

[33] V. Cardellini, V. Grassi, F. Lo Presti, M. Nardelli, Optimal operator placement for distributed stream processing applications, Journal of Parallel and Distributed Computing 99 (2016) 75–92. doi:10.1016/j.jpdc.2016.09.005.

[34] V. Cardellini, V. Grassi, F. Lo Presti, M. Nardelli, Decentralized self-adaptation for elastic data stream processing, Future Generation Computer Systems 87 (2018) 171–185. doi:10.1016/j.future.2018.05.025.

[35] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge intelligence: Paving the last mile of artificial intelligence with edge computing, Proceedings of the IEEE 107 (8) (2019) 1738–1762. doi:10.1109/JPROC.2019.2918951.

[36] Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, A survey on mobile edge computing: The communication perspective, IEEE Communications Surveys & Tutorials 19 (4) (2017) 2322–2358. doi:10.1109/COMST.2017.2745201.

[37] S. Taheri-abed, A. M. Eftekhari Moghadam, M. R. Meybodi, Machine learning-based computation offloading in edge and fog: A systematic review, Cluster Computing 26 (2023) 3495–3533. doi:10.1007/s10586-022-03798-7.

[38] T. Allaoui, K. Gasmi, T. Ezzedine, Reinforcement learning based task offloading of IoT applications in fog computing: Algorithms and

optimization techniques, Cluster Computing (2024). doi:10.1007/s10586-024-04308-3.

[39] E. M. Ali, J. Abawajy, F. Lemma, S. A. Baho, Analysis of deep reinforcement learning algorithms for task offloading and resource allocation in fog computing environments, Sensors 25 (17) (2025) 5286. doi:10.3390/s25175286.

[40] M. Nardelli, V. Cardellini, V. Grassi, F. Lo Presti, Efficient operator placement for distributed data stream processing applications, IEEE Transactions on Parallel and Distributed Systems 30 (8) (2019) 1753–1767. doi:10.1109/TPDS.2019.2896115.

[41] P. Pietzuch, J. Ledlie, J. Shneidman, M. Roussopoulos, M. Welsh, M. Seltzer, Network-aware operator placement for stream-processing systems, in: Proc. 22nd International Conference on Data Engineering (ICDE), 2006, p. 49. doi:10.1109/ICDE.2006.105.

[42] A. Akhter, M. Fragkoulis, A. Katsifodimos, Stateful functions as a service in action, in: Proc. VLDB Endowment, Vol. 12, 2019, pp. 1890–1893. doi:10.14778/3352063.3352092.

[43] M. R. Garey, D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, San Francisco, CA, 1979.

[44] gRPC Authors, gRPC: A high-performance, open source universal RPC framework (2023).
URL https://grpc.io

[45] CoreOS/etcd Authors, etcd: A distributed, reliable key-value store (2023).
URL https://etcd.io

[46] P. Virtanen, et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.

[47] L. Kleinrock, Queueing Systems, Volume 1: Theory, Wiley-Interscience, New York, NY, 1975.

[48] T. De Matteis, G. Mencagli, Keep calm and react with foresight: Strategies for low-latency and energy-efficient elastic data stream processing, ACM SIGPLAN Notices 52 (8) (2017) 539–553. doi:10.1145/3155284.3018709.

[49] S. Furuhashi, MessagePack: An efficient binary serialization format (2023).
URL https://msgpack.org

[50] A. Selivanov, et al., aiokafka: asyncio client for Kafka (2024).
URL https://github.com/aio-libs/aiokafka

[51] M. Placek, et al., aioboto3: Async boto3 wrapper for AWS SDK (2024).
URL https://github.com/terricain/aioboto3

[52] C. R. Harris, et al., Array programming with NumPy, Nature 585 (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[53] A. B. Kahn, Topological sorting of large networks, Communications of the ACM 5 (11) (1962) 558–562. doi:10.1145/368996.369025.

[54] Google, Protocol buffers (protobuf) (2024).
URL https://protobuf.dev

[55] S. Furuhashi, msgpack-python: MessagePack serializer implementation for Python (2024).
URL https://pypi.org/project/msgpack

[56] T. Saito, msgpack-java: MessagePack serializer implementation for Java (2024).
URL https://github.com/msgpack/msgpack-java

[57] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall/CRC, New York, NY, 1993. doi:10.1201/9780429246593.

[58] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83. doi:10.2307/3001968.